

Decoupling Variance and Scale-Invariant Updates in Adaptive Gradient Descent for Unified Vector and Matrix Optimization

Zitao Song¹ Cedar Site Bai¹ Zhe Zhang² Brian Bullins¹ David F. Gleich¹

Abstract

Adaptive methods like Adam have become the *de facto* standard for large-scale vector and Euclidean optimization due to their coordinate-wise adaptation with a second-order nature. More recently, matrix-based spectral optimizers like Muon (Jordan et al., 2024b) show the power of treating weight matrices as matrices rather than long vectors. Linking these is hard because many natural generalizations are not feasible to implement, and we also cannot simply move the Adam adaptation to the matrix spectrum. To address this, we reformulate the AdaGrad update and decompose it into a variance adaptation term and a scale-invariant term. This decoupling produces **DeVA** (**Decoupled Variance Adaptation**), a framework that bridges between vector-based variance adaptation and matrix spectral optimization, enabling a seamless transition from Adam to adaptive spectral descent. Extensive experiments across language modeling and image classification demonstrate that DeVA consistently outperforms state-of-the-art methods such as Muon and SOAP (Vyas et al., 2024), reducing token usage by around 6.6%. Theoretically, we show that the variance adaptation term effectively improves the blockwise smoothness, facilitating faster convergence. Our implementation is available at <https://github.com/Tsedao/Decoupled-Variance-Adaptation>

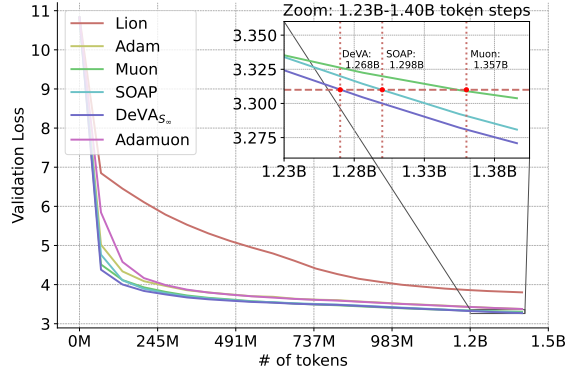


Figure 1. NanoGPT pretraining on FineWeb at a uniform 0.001 learning rate. Compared to non-adaptive Muon, the adaptive method SOAP and our method DeVA_s achieve the target validation perplexity using 4.3% and 6.6% fewer tokens, respectively.

1. Introduction

A fundamental acceleration strategy in modern optimization involves adapting stochastic gradient descent to second-order curvature (Amari, 1998). This adaptation enables accelerated progress along specific coordinates while suppressing exploding updates in others. Adaptive variants such as AdaGrad (Duchi et al., 2011), RMSprop (Tieleman, 2012), and Adam (Kingma & Ba, 2014) can be viewed as Euclidean gradient descent equipped with inexpensive approximations to curvature information. Among these, Adam has become the *de facto* standard for large-scale training, particularly for large language models (LLMs) (Dubey et al., 2024; Liu et al., 2024).

A simplifying aspect of adaptive gradient methods is that they ignore *matrix structure* within the parameters and treat them as a single long vector. Muon (Jordan et al., 2024b), a spectral gradient method for layerwise matrices, showed that adaptive gradient methods could be improved upon by using this matrix structure and treating matrix parameters as matrices within the optimization. Muon has since demonstrated superior LLM pretraining performance over Adam (Bai et al., 2025; Liu et al., 2025a; Wen et al., 2025). Although some may argue, we regard Muon as a non-adaptive method as we discuss below.

This presents an intriguing challenge: how do we combine

¹Department of Computer Science, Purdue University, West Lafayette, USA ²Edwardson School of Industrial Engineering, Purdue University, West Lafayette, USA. Correspondence to: Zitao Song <song903@purdue.edu>, David F. Gleich <dgleich@purdue.edu>.

Table 1. Special instantiations of Adaptive Subgradient descent $M^{-1}g$ and $M^{-1}G$ using preconditioner M^{-1} where $g \in \mathbb{R}^d$ and $G \in \mathbb{R}^{n \times m}$. Eigenbases are the rotation matrices we applied to the gradient G and Decoupled Variance is the adaptive stepsize we decoupled from sign functions in Equation (10). Here, σ_i is the i -th singular value of G , and $\odot$ is the Hadamard / elementwise product.

Method	Preconditioner M^{-1}	Eigenbases	Variance Adaptation
Adam (Kingma & Ba, 2014)	$\mathbb{E}[\text{diag}(gg^T)]^{-1/2}$	I	$(\text{EMA}[g^2]/g^2)^{-1/2}$
Shampoo (Gupta et al., 2018)	$(\mathbb{E}[GG^T]^{1/2} \otimes \mathbb{E}[G^T G]^{1/2})^{-1/2}$	I	None
SOAP (Vyas et al., 2024)	$((\mathbb{E}[GG^T] \otimes \mathbb{E}[G^T G]) / \text{Tr}(\mathbb{E}[GG^T]))^{-1/2}$	Q_L, Q_R	$(\text{EMA}[G \odot G] / G \odot G)^{-1/2}$
SPlus (Frans et al., 2025b)	$(\mathbb{E}[GG^T]^{1/2} \otimes \mathbb{E}[G^T G]^{1/2})^{-1/2}$	Q_L, Q_R	None
DeVA _{S∞} (Ours)	$(\mathbb{E}[(GG^T)^{1/2} \otimes (G^T G)^{1/2}]^{-1/2})^{-1/2}$	Q_L, Q_R	$(\text{EMA}[\sigma_i \sigma_j] / \sigma_i \sigma_j)^{-1/2}$

adaptive updates with matrix-based variables? Doing so efficiently should (and does!, see Figure 1) accelerate spectral descent, mirroring its impact on Euclidean gradient descent.

Efforts in this direction include AdaGO (Zhang et al., 2025), Adamuon (Si et al., 2025), and SPlus-then-Adam (Frans et al., 2025a), which attempt to incorporate Adam’s second-order moments directly into orthogonal updates in Muon. While this idea is pragmatic, a satisfactory theoretical framework for applying Adam’s adaptive stepsizes to matrices remains elusive. Consequently, it is essential for us to understand the origin of adaptive step sizes in vector-based updates and extend them to matrices smoothly.

In the context of vector updates, an interpretation from Balles & Hennig (2018); Orvieto & Gower (2025) characterizes Adam as an adaptive and smoothed variant of Signum (Bernstein et al., 2018). In this framework, the adaptive step size is modulated by the local *signal-to-noise* ratio (SNR). Stating this requires the standard m_t , which is the exponential moving average of the gradient that is used in Adam, as well as the term σ_t^2 , which is the exponential moving average of the variance $(m_{t-1} - g_t)^2$. These two terms allow the Adam update to be written as¹

$$\Delta x_t = - \underbrace{\frac{\eta}{\sqrt{1 + \sigma_t^2/m_t^2}}}_{\text{adaptive stepsize}} \underbrace{\text{sign}(m_t)}_{\text{direction}}. \quad (1)$$

This formulation allows a decoupling of the direction and the adaptive stepsize. Crucially, the gradient variance term σ_t^2 only arises in the stepsize.

The Muon update can be written as a non-adaptive update with the *matrix sign* function (Chen et al., 2025), which we call msign . Analytically, the particular form used involves applying the *sign* function to the vector of singular values in an SVD and then recombining. The result is an equally weighted set of all the orthogonal directions in the data

¹Here, η is a standard learning rate step size that applies to all coordinates. In future sections we drop this piece of the update.

(see Equation (8)). In Muon,

$$\Delta X_t = -\eta \text{msign}(M_t), \quad (2)$$

where M_t is a first-order and extra Nesterov momentum combination of gradient matrices. According to Jordan et al. (2024b), the choice of msign here is hypothesized to help by amplifying “rare directions”, those with small singular values that are nonetheless critical for learning. More recently, Davis & Drusvyatskiy (2025) shows that msign will be helpful when the matrix gradient G_t has a large Nuclear-to-Frobenius norm ratio.

In the context of these two expressions for Adam and Muon, we wish to explore whether one can derive an adaptation matrix Γ_t that *functions as* the adaptive stepsize in Adam but works for a matrix spectrum. Formally, we want to ask can we derive an update

$$\Delta X_t = - \underbrace{\eta \Gamma_t}_{\text{matrix adaptive stepsize}} \underbrace{\odot \text{msign}(G_t)}_{\text{matrix directions}}, \quad (3)$$

where $\odot$ is the Hadamard product?

In this work, we systematically address this idea by introducing **DeVA** (**Decoupled Variance Adaptation**), a unified framework that interprets adaptive gradient methods as preconditioned gradient descent. As a result, popular algorithms could be viewed as special instances of the general framework with different instantiations of the preconditioners M (see Table 1). Based on that, **DeVA** derives coordinate-wise adaptive stepsizes naturally for matrix parameters by decoupling a gradient variance adaptation and a scale-invariant update from the underlying preconditioned update. Our contributions are threefold:

Unification: Under a diagonal curvature approximation we derive $\text{DeVA}_{\ell_\infty}$ and establish its fundamental connection to Adam. Extending this to structured matrix updates, we utilize a Kronecker approximation to introduce DeVA_{S_∞} , a novel adaptive spectral gradient descent method that enables curvature-aware updates throughout the training process (see Theorem 3.1). This framework further generalizes to other preconditioned methods, such as Shampoo (Gupta et al., 2018), SOAP (Vyas et al., 2024), and SPlus (Frans

et al., 2025b) when the expectation of the Kronecker product is separable.

Numerical Validation: Our experiments, ranging from synthetic optimization benchmarks to language modeling (see Figure 1) and large-scale image classification (see Figure 4) demonstrate that DeVA_{S_∞} consistently outperforms its non-adaptive counterpart Muon and adaptive spectral descent competitors SOAP and Adamuon.

Theoretical Implication: Finally, under the blockwise smoothness assumption, our theoretical analysis justifies the necessity of incorporating adaptive stepsize into matrix-based optimizers by showing it helps to reduce the blockwise smoothness that appears inside the convergence result (see Theorem 4.12 and Theorem 4.15).

Organization. Section 2 provides preliminaries and key related work on descent methods. Sections 3 and 4 introduce and analyze the DeVA framework and its variants ($\text{DeVA}_{\ell_\infty}$, DeVA_{S_∞}). Numerical evaluations follow in Section 5, while additional related work is delegated to Appendix A.

2. Preliminaries

Notation. We use lowercase italics (x, g) for vectors/scalars and uppercase letters (X, G) for matrices. Powers are *matrix-powers*, for example, $X^{-1/2}$ denotes the matrix inverse square root unless explicitly stated otherwise. Subscripts x_t denote iterates, and $\text{EMA}[\cdot]$ signifies the exponential moving average. We denote the inner product as $\langle \cdot, \cdot \rangle$, while $\odot$, $\oslash$, and $\otimes$ represent the Hadamard product, Hadamard division, and Kronecker product, respectively. We use e_i as the i -th standard basis vector, and $\mathbf{1}_m$ is the m -dimensional all-ones vector. Where clear from context, we'll use g_t to be the function gradient at the t th iteration for the vector case and G_t to be the function gradient for a matrix variable.

Steepest Gradient Descent. Given a general function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and a Mahalanobis norm defined on a positive-definite matrix M , i.e., $\|\cdot\|_M = \sqrt{\langle \cdot, M \cdot \rangle}$, let $g = \nabla f(x)$. Then steepest gradient descent under this norm generates a direction Δx^* by minimizing a local quadratic approximation of f at x :

$$\Delta x^* = \arg \min_{\Delta x} \langle g, \Delta x \rangle + \frac{1}{2} \|\Delta x\|_M^2. \quad (4)$$

Solving this yields $\Delta x^* = -M^{-1}g$, where M^{-1} is often referred to as a *preconditioner*. This formulation is the basis for adaptive subgradient methods (Duchi et al., 2011).

In AdaGrad (Duchi et al., 2011), M is defined as $(\sum_t g_t g_t^T)^{1/2}$, a choice motivated by regret minimization. We will see many gg^T operations and it simplifies notation

to let $H = gg^T$. More recently, M has been interpreted as a *whitening metric* (Frans et al., 2025b; Eschenhagen et al., 2026), defined as the square root of the gradient covariance, $M_{\text{Whitening}} := \mathbb{E}[H]^{1/2}$ (Unless stated otherwise, $\mathbb{E}[\cdot]$ denotes expectation over minibatches), so that $M_{\text{Whitening}}^{-1}$ normalizes the covariance of the gradient vector g . This yields the whitening update

$$\Delta x_{\text{whitening}}^* = -\mathbb{E}[H]^{-1/2}g. \quad (5)$$

The setup leads to some nice connections such as an equivalence with the Fisher Information Matrix $F(x)$ when f is a negative log-likelihood function, in which case the Hessian $\mathcal{H}(x) := \nabla^2 f(x)$ also coincides with $F(x)$. Under this scenario, the $\mathbb{E}[H]$ term can further be considered as the Gauss-Newton or the covariance matrix of the gradient (Martens, 2020; Morwani et al., 2024). Notably, Adam (Kingma & Ba, 2014) (or RMSprop (Tieleman, 2012) with zero first-order momentum) can be viewed as a computationally efficient realization of Equation (5), where H is restricted to a diagonal approximation $\text{diag}(gg^T)$ and the expectation is replaced by an exponential moving average (EMA). This avoids the intractability of computing matrix square roots in high-dimensional parameter spaces.

Matrix-based Descent. For a function $f : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}$ with parameter $X \in \mathbb{R}^{n \times m}$ and gradient $G \in \mathbb{R}^{n \times m}$, a standard AdaGrad-style update requires maintaining a prohibitively large $H \in \mathbb{R}^{nm \times nm}$ by vectorizing G . To circumvent this, Shampoo (Gupta et al., 2018) approximates the covariance matrix $\mathbb{E}[\text{vec}(G) \text{vec}(G)^T]$ by accumulating two smaller factorized covariance matrices, $\mathbb{E}[L] \in \mathbb{R}^{n \times n}$ and $\mathbb{E}[R] \in \mathbb{R}^{m \times m}$, where $L = GG^T$ and $R = G^T G$. The resulting Shampoo update ΔX^* , expressed in the form of Equation (5), is given by:

$$\text{vec}(\Delta X_{\text{Shampoo}}^*) = -\left(\mathbb{E}[L]^{1/2} \otimes \mathbb{E}[R]^{1/2}\right)^{-1/2} \text{vec}(G). \quad (6)$$

Similarly, SOAP (Vyas et al., 2024; Morwani et al., 2024) approximates the Gauss-Newton matrix by computing an optimal Kronecker product approximation, which yields:

$$\text{vec}(\Delta X_{\text{SOAP}}^*) = -((\mathbb{E}[L] \otimes \mathbb{E}[R]) / \text{Tr}(\mathbb{E}[L]))^{-1/2} \text{vec}(G). \quad (7)$$

If we consider the gradient sample by removing the expectation in Equation (6), the term $(L^{1/2} \otimes R^{1/2})^{-1/2} \text{vec}(G)$ can be simplified. Using the identity $(A \otimes B)^k = A^k \otimes B^k$ and the vectorization property $(A \otimes B) \text{vec}(X) = \text{vec}(AXB^T)$, the update simplifies to $\Delta X^* = -L^{-1/4}GR^{-1/4}$. Substituting the singular value decomposition $G = U\Sigma V^T$, the update becomes:

$$-\Delta X_{\text{SpectralGD}}^* = (GG^T)^{-1/4}G(G^TG)^{-1/4} = UV^T. \quad (8)$$

This result recovers spectral gradient descent methods (Carlson et al., 2015a; Jordan et al., 2024b), which effectively orthogonalize the gradient via the matrix sign function. Consequently, by setting $H = L^{1/2} \otimes R^{1/2}$, update $-H^{-1/2}\text{vec}(G)$ indeed is performing an orthogonalization step. If we plug this H back to the whitening update in Equation (5), we can reframe it as a Shampoo-style update:

$$\text{vec}(\Delta X_{\text{Shampoo-style}}^*) = -\mathbb{E}[L^{1/2} \otimes R^{1/2}]^{-1/2} \text{vec}(G). \quad (9)$$

3. Our Methods

In this section, we present the **Decoupled Variance Adaptation (DeVA)** framework. To motivate our approach, consider the whitening update in Equation (5): $\Delta x^* = -\mathbb{E}[H]^{-1/2}g$. As established in previous sections, if $H = \text{diag}(gg^T)$, $\mathbb{E}[H]$ becomes, under the EMA approximation, the adaptive step size in Adam and RMSprop. In the matrix case where $H = L^{1/2} \otimes R^{1/2}$, $\mathbb{E}[H]$ then becomes the Shampoo-style update.

If we remove the expectation of H , we make a key observation. Specifically, $H^{-1/2}g$ yields a sign-based update: it recovers $\text{sign}(g)$ when $H = \text{diag}(gg^T)$ and the matrix sign $\text{msign}(G)$ when $H = L^{1/2} \otimes R^{1/2}$. This suggests that the scale-invariant update, $\text{sign}(g)$ or $\text{msign}(g)$, is indeed embedded within the preconditioned update $H^{-1/2}g$.

Motivated by that, we define the DeVA framework by refactoring the whitening update in Equation (5) as follows:

$$\Delta x_{\text{whitening}}^* = - \underbrace{\left(\mathbb{E}[H]^{-1/2}H^{1/2}\right)}_{\text{Decoupled Variance Adaptation}} \cdot \underbrace{\left(H^{-1/2}g\right)}_{\text{Scale-Invariant update}}, \quad (10)$$

This decomposition provides an appealing representation of adaptive step sizes for gradient descent methods with a scale-invariant update. Here we call the term $\mathbb{E}[H]^{-1/2}H^{1/2}$ as *variance adaptation*. When $H = gg^T$, this *variance adaptation* is also called the *signal-to-noise ratio* (SNR) of g in Balles & Hennig (2018); Orvieto & Gower (2025), i.e., $(gg^T/\mathbb{E}[gg^T])^{1/2}$, where a larger SNR implies a larger update in the steepest decent direction. In the following subsections, we first establish that the DeVA framework recovers Adam in the vector setting; we then generalize this framework to matrix optimization, introducing a novel adaptive spectral descent method.

3.1. Connection to Adam

For vectors $\Delta x, g \in \mathbb{R}^d$, let $H = \text{diag}(gg^T)$. We can express the DeVA framework from Equation (10) in an

Algorithm 1 DeVA $_{\ell_\infty}$

Input $\beta_1, \beta_2, \varepsilon, T, \eta_t$ for $t = 1, \dots, T$

Output $x_T \in \mathbb{R}^d$

- 1: Initialize m_0, v_0 to 0
 - 2: Randomly initialize $x_1 \in \mathbb{R}^d$
 - 3: **for** $t = 1, \dots, T$ **do**
 - 4: $g_t = \nabla f(x_t, \xi_t)$
 - 5: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$
 - 6: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)m_t^2 \leftarrow \text{differs from Adam}$
 - 7: $\gamma_t = \left(\frac{v_t}{m_t^2 + \varepsilon}\right)^{-1/2}$
 - 8: $d_t = \gamma_t \odot \text{sign}(m_t)$
 - 9: $x_{t+1} = x_t - \eta_t d_t$
 - 10: **end for**
-

element-wise form as:

$$\Delta x_{\text{DeVA}_{\ell_\infty}}^* = - \underbrace{\left(\frac{\mathbb{E}[g^2]}{g^2}\right)^{-1/2}}_{\text{Decoupled Variance Adaptation}} \odot \text{sign}(g), \quad (11)$$

where g^2 denotes the element-wise square and division is element-wise. In this formulation, we have successfully decoupled the gradient variance term from the standard sign-based update. Since $\text{sign}(g)$ is the solution to the constrained linear minimization problem under an ℓ_∞ -norm ball, i.e., $\text{sign}(g) := -\arg \min_{\|\Delta x\|_{\ell_\infty} \leq 1} \langle g, \Delta x \rangle$, we refer to this specific instance as DeVA $_{\ell_\infty}$.

Notably, DeVA $_{\ell_\infty}$ recovers the Adam optimizer when the gradient expectation is approximated by the EMA under the bias correction (Kingma & Ba, 2014). Recall that the Adam update (or RMSprop with zero first-order momentum) is given by:

$$\Delta x_{\text{Adam}}^* = -\frac{g}{\sqrt{\text{EMA}[g^2]}} = -\left(\frac{\text{EMA}[g^2]}{g^2}\right)^{-1/2} \odot \text{sign}(g). \quad (12)$$

When first-order momentum is incorporated, Adam replaces the raw gradient g with its moving average $m_t = \text{EMA}[g]$. Similarly, in the DeVA framework, we replace g with its expectation $\mathbb{E}[g]$ in Equation (11). In practice, since evaluating the true expectation $\mathbb{E}[g]$ is computationally prohibitive, DeVA $_{\ell_\infty}$ also utilizes the $\text{EMA}[g]$ to approximate $\mathbb{E}[g]$.

A key distinction to Adam arises in this substitution: by replacing g with m_t in Equation (11), the numerator of the decoupled variance term becomes $\mathbb{E}[m_t^2]$. This implies that we are accumulating the square of the expected gradient to estimate the second-order moment, rather than the squared sample gradient (Line 6 in Algorithm 1). This shift also aligns with SNR interpretation (Orvieto & Gower, 2025), where the update magnitude is modulated by the reliability

Algorithm 2 DeVAS_∞

Input $\beta_1, \beta_2, \beta_3, \text{freq}, \varepsilon, T, \eta_t$ for $t = 1, \dots, T$
Output $X_T \in \mathbb{R}^{n \times m}$

- 1: Initialize L_0, R_0, M_0, V_0 to 0
- 2: Randomly initialize $X_1 \in \mathbb{R}^{n \times m}$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $G_t = \nabla f(X_t, \xi_t)$
- 5: $L_t = \beta_3 L_{t-1} + (1 - \beta_3) G_t G_t^T$
- 6: $R_t = \beta_3 R_{t-1} + (1 - \beta_3) G_t^T G_t$
- 7: **if** $t \bmod \text{freq}$ is 0 **then**
- 8: $Q_L, Q_R = \text{eigdecomp}(L_t), \text{eigdecomp}(R_t)$
- 9: **end if**
- 10: $G'_t = Q_L^T G_t Q_R$
- 11: $M_t = \beta_1 M_{t-1} + (1 - \beta_1) G'_t$
- 12: $r_t = \left(\sqrt{\sum_{j=1}^m (M_t)_{ij}^2} \right)_{i=1}^n$
- 13: $c_t = \left(\sqrt{\sum_{i=1}^n (M_t)_{ij}^2} \right)_{j=1}^m$
- 14: $V_t = \beta_2 V_{t-1} + (1 - \beta_2) r_t c_t^T \leftarrow \text{spectral adaptivity}$
- 15: $\Gamma_t = (V_t \odot (r_t c_t^T + \varepsilon))^{-1/2}$
- 16: $D_t = \Gamma_t \odot \text{msign}(M_t) \odot 0.2 \sqrt{\max(n, m)}$
- 17: $X_{t+1} = X_t - \eta_t Q_L D_t Q_R^T$
- 18: **end for**

of the gradient signal m_t relative to its stochastic variance $\text{Var}(m_t)$. The complete procedure for DeVA_{ℓ_∞} is summarized in Algorithm 1.

3.2. Matrix Extension

We now extend our framework to the matrix case, where $X, G \in \mathbb{R}^{n \times m}$. Here we let $H = L^{1/2} \otimes R^{1/2}$, where $L = G G^T \in \mathbb{R}^{n \times n}$ and $R = G^T G \in \mathbb{R}^{m \times m}$ are the left and right factorized covariance matrices, respectively. For $\Delta X \in \mathbb{R}^{n \times m}$, the DeVA representation yields:

$$\begin{aligned}
& \text{vec}(\Delta X_{\text{DeVAS}_\infty}^*) \\
&= -\mathbb{E}[H]^{-1/2} H^{1/2} H^{-1/2} \text{vec}(G) \\
&= -\underbrace{\mathbb{E}[(L \otimes R)^{1/2}]^{-1/2} (L \otimes R)^{1/4}}_{\text{Decoupled Variance Adaptation}} \text{vec}(\text{msign}(G)).
\end{aligned} \tag{13}$$

where the second equality follows $H^{-1/2} \text{vec}(G)$ gives a msign update when $H = L^{1/2} \otimes R^{1/2}$. Since $\text{msign}(G)$ is the solution of steepest gradient descent under Schatten- ∞ (S_∞) norm, we name this variant as DeVAS_∞. While Equation (13) successfully isolates the matrix sign function, the decoupled variance term remains computationally intractable due to the high-dimensional matrix powers involved. To resolve this, we leverage the eigendecomposition approach used in SOAP (Vyas et al., 2024), allowing us to derive the equivalent but more efficient spectral representation presented in Theorem 3.1.

Theorem 3.1 (Coordinate-wise DeVAS_∞). *Let $L \in \mathbb{R}^{n \times n}$*

and $R \in \mathbb{R}^{m \times m}$ be Kronecker factors with eigendecompositions $L = Q_L \Lambda_L Q_L^T$ and $R = Q_R \Lambda_R Q_R^T$. Let $\sigma_i = \sqrt{\lambda_i}$ and $\sigma_j = \sqrt{\mu_j}$ denote the singular values corresponding to the eigenvalues of L and R , respectively. With the Kronecker curvature approximation $\mathbb{E}[L^{1/2} \otimes R^{1/2}]$, and assuming local stability (see the proof), the optimal update ΔX^ in Equation (10) has spectral coordinate-wise form:*

$$\Delta X_{\text{DeVAS}_\infty}^* = -Q_L \left(\tilde{E}^{-1/2} \odot \text{msign}(Q_L^T G Q_R) \right) Q_R^T, \tag{14}$$

where $\tilde{E} \in \mathbb{R}^{n \times m}$ is the spectral adaptation matrix with entries defined by the ratio of expected to sample gradient singular value products:

$$\tilde{E}_{ij} = \frac{\mathbb{E}[\sigma_i \sigma_j]}{\sigma_i \sigma_j}. \tag{15}$$

Here, $\odot$ and the exponent $-1/2$ denote element-wise operations within the rotated spectral space.

Proof Sketch: The derivation proceeds in three main steps:

1. **Spectral Decomposition:** Under the assumption of local stability, the expectation $\mathbb{E}[L^{1/2} \otimes R^{1/2}]$ factorizes as $(Q_L \otimes Q_R) \mathbb{E}[\Lambda_L^{1/2} \otimes \Lambda_R^{1/2}] (Q_L^T \otimes Q_R^T)$.
2. **Vectorization and Commutation:** Using the identity $(A \otimes B) \text{vec}(X) = \text{vec}(A X B^T)$ and the fact that orthogonal transformations commute with the msign (i.e., $Q \text{msign}(X) = \text{msign}(Q X)$), we project the update into the spectral space defined by Q_L and Q_R .
3. **Diagonal Compression:** We define the diagonal scaling matrix $E = \mathbb{E}[(\Lambda_L \otimes \Lambda_R)^{1/2}] (\Lambda_L \otimes \Lambda_R)^{-1/2}$ and compress the $nm \times nm$ diagonal matrix E into the $n \times m$ matrix $\tilde{E}$, where $\tilde{E}_{ij} = \mathbb{E}[\sigma_i \sigma_j] / (\sigma_i \sigma_j)$.

Substituting these into the vectorized update and mapping back to matrix form yields the Hadamard product $\Delta X^* = -Q_L (\tilde{E}^{-1/2} \odot \text{msign}(Q_L^T G Q_R)) Q_R^T$. $\square$

The coordinate-wise simplification in Theorem 3.1 reveals three significant insights. First, the numerator $\mathbb{E}[\sigma_i \sigma_j]$ tracks the covariance of the singular values of the gradient, rather than the raw gradient covariance $\mathbb{E}[g g^T]$ used in standard vector adaptive methods. Second, the variance adaptation and the matrix sign update occur within the rotated spectral space defined by Q_L and Q_R , which provides a principled choice of basis. Finally, the Kronecker curvature approximation can be reduced to an inexpensive element-wise multiplication, facilitating efficient implementations on large-scale problems. The complete derivation is provided in Appendix C.1.

3.3. Practical Implementation

In the DeVA_{S_∞} framework, the decoupled variance matrix is composed of entries $\frac{\mathbb{E}[\sigma_i \sigma_j]}{\sigma_i \sigma_j}$. However, the numerator requires the expectation of the singular values of the gradient G at each iteration, which is still computationally prohibitive. To address this, we remark on the following connection between the eigenvalues of the factorized covariance matrices and the rotated gradient G' .

Proposition 3.2. *For a gradient $G \in \mathbb{R}^{n \times m}$, let $Q_L \Lambda_L Q_L^T$ and $Q_R \Lambda_R Q_R^T$ be the eigendecompositions of $L = GG^T$ and $R = G^T G$, respectively. Let λ_i and μ_j denote the i -th and j -th eigenvalues of L and R . If we define rotated gradient as $G' = Q_L^T L G Q_R$, then:*

$$\lambda_i = e_i^T (G' \odot G') 1_m, \quad \mu_j = 1_n^T (G' \odot G') e_j. \quad (16)$$

Corollary 3.3. *Let $G' = Q_L^T L G Q_R$. Let σ_i be the i -th singular value of G and $\text{rank}(G) = r$, for $i, j \leq r$, we have*

$$\sigma_i = \sqrt{\lambda_i} = \|G'_{i,:}\|_2, \text{ or } \sigma_j = \sqrt{\mu_j} = \|G'_{:,j}\|_2,$$

where $\|G'_{i,:}\|_2$ and $\|G'_{:,j}\|_2$ denote the ℓ_2 -norms of the i -th row and j -th column of G' , respectively.

Note that both of these are fancy ways of using the singular value decomposition of G , which gives G' as the diagonal matrix of singular values. We state them, however, because they motivate the final update we use as a computable approximation. Equipped with Corollary 3.3, we can represent the expectation, $\mathbb{E}[\sigma_i \sigma_j]$, as the entries of the matrix $\mathbb{E}[rc^T]$, where $r \in \mathbb{R}^n$, $c \in \mathbb{R}^m$, $r_i = \|G'_{i,:}\|_2$, and $c_j = \|G'_{:,j}\|_2$.

This structure bears a resemblance to the second-order moment in the idealized AdaFactor (Shazeer & Stern, 2018) update discussed in SOAP (Vyas et al., 2024). However, a key distinction is that idealized SOAP estimates the second momentum by taking the product of independent expectations of squared norm and then normalize, i.e., $(\mathbb{E}[r^2] \mathbb{E}[c^2]^T) \odot \mathbb{E}[r^2]$, whereas DeVA_{S_∞} take the expectation of the outer product of unsquared norm, i.e., $\mathbb{E}[rc^T]$. Generally, to compute the preconditioner M , the expectation of a Kronecker product in DeVA_{S_∞} does not equal the Kronecker product of expectations in SOAP. This discrepancy marks a fundamental distinction between our work and existing methods such as K-FAC (Martens & Grosse, 2015) and Shampoo (Gupta et al., 2018).

To extend DeVA_{S_∞} to momentum-based optimization, we apply an EMA to the rotated gradient G' and detail the complete DeVA_{S_∞} procedure in Algorithm 2. We emphasize that since we accumulate the moving average of the rotated gradient $\text{EMA}[G']$ not G' itself, the second momentum should track $\text{EMA}[rc^T]$ (Line 12–14). Furthermore, following Bai et al. (2025), we normalize the orthogonalized update by $0.2\sqrt{\max(n, m)}$ (Line 16) to align the RMS

norm of standard Adam updates. Finally, to maintain computational efficiency, we approximate the full eigendecomposition (Line 8) with a single-step power iteration and QR decomposition, as suggested by Vyas et al. (2024).

4. Analysis

To characterize the updates, we first define a norm that can incorporate the variance adaptation weights.

Definition 4.1 (γ -Weighted Dual Norm). Let $\|\cdot\|$ be a norm on $\mathbb{R}^n$ and $\langle \cdot, \cdot \rangle$ be the standard inner product. Given a weight vector $\gamma \in \mathbb{R}_{++}^n$, we define the γ -weighted inner product as $\langle s, x \rangle_\gamma := \langle s \odot \sqrt{\gamma}, x \odot \sqrt{\gamma} \rangle$. Let $W = \text{diag}(\gamma)$, the γ -weighted dual norm $\|\cdot\|_{*,\gamma}$ is defined as:

$$\|s\|_{*,\gamma} := \sup_{\|x\| \leq 1} \langle s, x \rangle_\gamma = \|Ws\|_* \quad (17)$$

Recall that for any ℓ_p -norm, the solution to the constrained linear minimization problem $\arg \min_{\|x\| \leq 1} \langle s, x \rangle$ provides the steepest descent direction. Specifically, for $i = 1, \dots, d$, the optimal components are given by $x_i^* = -\text{sign}(s_i) |s_i|^{q-1} / \|s\|_q^{q-1}$ where $1/p + 1/q = 1$. We now provide concrete examples for the norms used in our framework.

Example 4.2 (ℓ_∞ -norm). For the ℓ_∞ -norm, given weights $\gamma_i \geq 0$ for $i = 1, \dots, d$, we have $\|s\|_{*,\gamma} = \|\gamma \odot s\|_1 = \sum_{i=1}^d \gamma_i |s_i|$.

Example 4.3 (S_∞ -norm). For $X \in \mathbb{R}^{n \times m}$, let $\|X\| = \sigma_1(X)$ (S_∞ -norm), $\|S\|_1 = \text{Tr}(\sqrt{S^T S})$, and $\langle S, X \rangle = \text{Tr}(S^T X)$. For a positive weight matrix $\Gamma \in \mathbb{R}_{++}^{n \times m}$, we have $\|S\|_{*,\Gamma} = \|\Gamma \odot S\|_1$. The maximizer is the orthogonal matrix UV^T from the SVD of $(\Gamma \odot S)$.

Example 4.4 (Preconditioned matrix seminorm (Veprikov et al., 2025)). For $X \in \mathbb{R}^{n \times m}$, when Γ is a diagonal positive matrix $D \in \mathbb{R}^{n \times m}$, preconditioned matrix seminorm $\|S\|_{*,D} := \|D \odot S\|_*$ is a special instance of Γ -weighted dual norm.

Example 4.5 (Nuclear rank of S (Davis & Drusvyatskiy, 2025)). For $X \in \mathbb{R}^{n \times m}$, when Γ is a diagonal positive matrix $D \in \mathbb{R}^{n \times m}$ where $D_{ii} = 1/\|S\|_F$ for all i , $\|S\|_{*,D}^2 = \|S\|_*^2 / \|S\|_F^2$ is the nuclear rank of S .

Remark 4.6. When $S = \nabla f(X)$, the nuclear rank measurement (Example 4.5) serves as a theoretical indicator for when spectral descent outperforms standard gradient descent (Davis & Drusvyatskiy, 2025; Shen et al., 2025).

Remark 4.7. In the S_∞ -norm case of Example 4.3, we have $\min(\Gamma) \|S\|_1 \leq \langle S, \text{msign}(S) \rangle_\Gamma \leq \langle S, \text{msign}(\Gamma \odot S) \rangle_\Gamma = \|S\|_{1,\Gamma}$.

Remark 4.7 justifies that we can consider $\langle S, \text{msign}(\Gamma \odot S) \rangle_\Gamma$ as a valid measure for the stationarity condition of the matrix-valued function in Theorem 4.15.

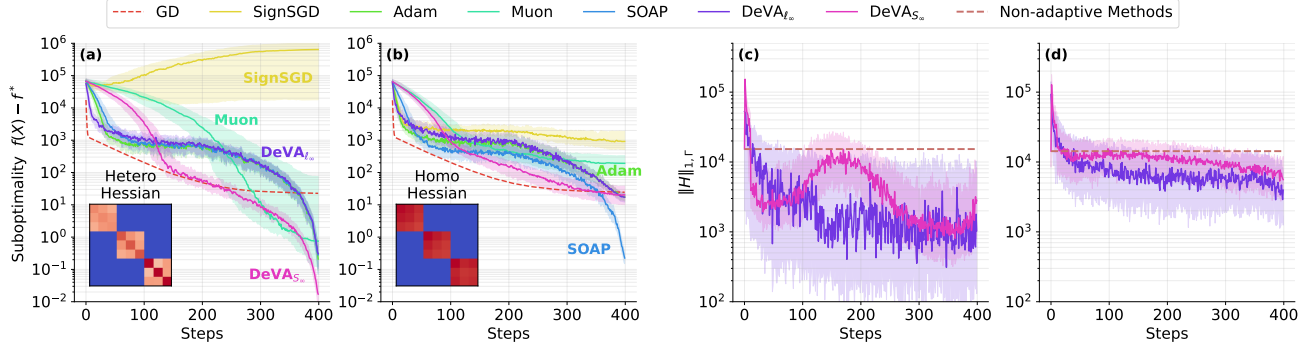


Figure 2. Trace Quadratic Function Optimization (median and 25%/75% quantiles over 100 seeds). (a)–(b): Training performance for various optimizers on 9-dimensional trace quadratic problems with homogeneous vs. heterogeneous Hessians (details see Appendix B). DeVA $_{S_\infty}$ significantly outperforms Muon in the heterogeneous setting. (c)–(d): Dynamic of weighted dual norm $\|H\|_{1,\Gamma}$ (see Definition 4.1) for adaptive methods. Adaptive methods (DeVA $_{\ell_\infty}$, DeVA $_{S_\infty}$) effectively reduce $\|H\|_{1,\Gamma}$ as predicted by Theorems 4.12 and 4.15, with a more pronounced reduction in the heterogeneous case (c) compared to the homogeneous case (d).

4.1. Convergence result for DeVA $_{\ell_\infty}$

To facilitate analysis, we consider a simplified version of the vector-case of DeVA $_{\ell_\infty}$ where the momentum terms are zero ($\beta_1 = \beta_2 = 0$), resulting in the following update rule:

$$x_{t+1} = x_t - \eta \gamma_t \odot \text{sign}(g_t). \quad (18)$$

We first define the smoothness geometry below.

Assumption 4.8 (Blockwise Smoothness). The objective function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is blockwise L_v -smooth. Specifically, there exists a partition of the d dimensions into c blocks and a smoothness vector $L_v = [L_{v,1}, \dots, L_{v,c}] \in \mathbb{R}_{++}^c$ such that for any $x, y \in \mathbb{R}^d$ we have $f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{1}{2} \sum_{i=1}^c L_{v,i} \|y_i - x_i\|_2^2$, where x_i is the i -th block.

The blockwise smoothness assumption addresses the blockwise diagonal structure in Hessian frequently observed in neural networks (Ghorbani et al., 2019; Zhang et al., 2024b). This Hessian heterogeneity means that the standard choice of step size—namely, the inverse of a single global smoothness constant used in convex smooth analysis (Nesterov et al., 2018)—is no longer optimal, motivating instead a blockwise step size that adapts to the local curvature of each block.

Remark 4.9. For notation consistency, we broadcast $L_{v,i}$ to each block to match the dimension of $\mathbb{R}^d$ in the later analysis.

Assumption 4.10. Given mini-batch stochasticity ξ and stochastic gradient $g := \nabla f(x, \xi) \in \mathbb{R}^d$, we assume an unbiased estimator $\mathbb{E}[g] = \nabla f(x)$. Furthermore, for any $i = 1, \dots, d$, the variance is bounded such that $\mathbb{E}[(g_i - \nabla f(x_i))^2] \leq \sigma_i^2$.

Remark 4.11. In our analysis, we utilize a mini-batch stochastic gradient $g_t = \frac{1}{n_t} \sum_{j=1}^{n_t} \nabla f(x_t, \xi_j)$. This reduces the effective variance per block to σ_i^2/n_t . We use $\mathcal{F}_t := \sigma(\xi_1, \dots, \xi_t)$ to denote the natural filtration.

Theorem 4.12. Under Assumption 4.8 and Assumption 4.10, for $\sigma_v, L_v, \gamma_t \in \mathbb{R}^d$, let $\Delta = f(x_1) - f^*$, for a fixed learning rate $\eta = 1/\sqrt{T}$, a mini-batch size $n_t = T$, suppose the weight process $\{\gamma_t\}_{t=0}^d$ is bounded and $\mathcal{F}_{t-1}$ measurable, let $\hat{\gamma} = \frac{1}{T} \sum_{t=1}^T \gamma_t$ and $\hat{\gamma}^2 = \frac{1}{T} \sum_{t=1}^T \gamma_t^2$, the iterates using Equation (18) satisfy:

$$\frac{1}{T} \sum_{t=1}^T \|\nabla f(x_t)\|_{1,\gamma_t} \leq \frac{1}{\sqrt{T}} \left(\Delta + 2\|\sigma_v\|_{1,\hat{\gamma}} + \frac{1}{2}\|L_v\|_{1,\hat{\gamma}^2} \right).$$

Theorem 4.12 generalizes the non-adaptive guarantees of (Bernstein et al., 2018) by replacing the standard smoothness error $\|L_v\|_1$ with a weighted term $\|L_v\|_{1,\hat{\gamma}^2}$. This new metric quantifies the alignment between blockwise smoothness and our curvature estimate, where $\hat{\gamma}_i^2$ corresponds to the i -th diagonal element of the inverse Fisher information approximation, $\mathbb{E}[\text{diag}(gg^T)]^{-1}$.

Unlike standard SignSGD (where $\gamma_t = 1_d$), our decoupled adaptive approach uses a weight process $\gamma_t \propto \mathbb{E}[\text{diag}(g_t g_t^T)]^{-1/2}$ to actively scale the smoothness parameters. In the case of $\hat{\gamma}_i^2 \leq 1$, the error term $\hat{\gamma}_i^2 L_{v,i}$ effectively reduces the impact of blockwise smoothness, potentially lowering the total error $\|L_v\|_{1,\hat{\gamma}^2}$ toward the dimension d in optimal cases. Full proofs are available in Appendix D.1.

4.2. Convergence result for DeVA $_{S_\infty}$

Now turn our attention to the matrix case and we consider the same type of simplified version of the DeVA $_{S_\infty}$ update rule with zero momentum ($\beta_1 = \beta_2 = \beta_3 = 0$):

$$X_{t+1} = X_t - \eta \Gamma_t \odot \text{msign}(G_t). \quad (19)$$

To analyze this matrix update, we extend the concept of blockwise smoothness to the matrix manifold.

Assumption 4.13. For any matrices $X, Y \in \mathbb{R}^{n \times m}$, there exists a smoothness matrix $L_m \in \mathbb{R}_{++}^{n \times m}$ such that $f(Y) \leq$

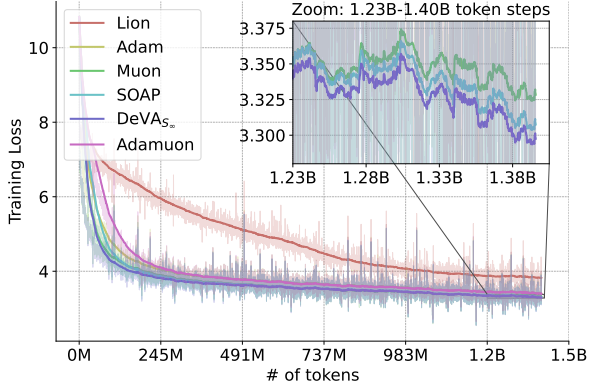


Figure 3. EMA-smoothed training loss for a 274M NanoGPT model on FineWeb. All methods use a uniform learning rate of 0.001. DeVAs_∞ consistently outperforms baseline optimizers throughout the training duration.

$f(X) + \langle \nabla f(X), Y - X \rangle + \frac{1}{2} \|\sqrt{L_m} \odot (Y - X)\|_F^2$, where $\|\cdot\|_F$ denotes the Frobenius norm and $\sqrt{L_m}$ is the element-wise square root of L_m .

Remark 4.14. Assumption 4.13 can be viewed as a specific instance of the blockwise smoothness in Assumption 4.8, where each entry of the matrix is treated as an individual block ($c = nm$).

Theorem 4.15. Under Assumption 4.13 and Assumption 4.10, for $\sigma_m, L_m, \Gamma_t \in \mathbb{R}^{n \times m}$, let $r = \min(n, m)$ and $\Delta = f(X_1) - f^*$, for a learning rate $\eta = 1/\sqrt{T}$ and mini-batch size $n_t = T$, suppose the weight process $\{\Gamma_t\} \subset \mathbb{R}_{\geq 0}^{n \times m}$ is bounded and $\mathcal{F}_{t-1}$ measurable, let $\hat{\Gamma} = \sqrt{\frac{1}{T} \sum_{t=1}^T \Gamma_t^2}$ and $\hat{\Gamma}^2 = \frac{1}{T} \sum_{t=1}^T \Gamma_t^2$. Defining the stationarity measure $\psi(G_t) = \mathbb{E}[\langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t}]$, the iterates using Equation (19) satisfy:

$$\frac{1}{T} \sum_{i=1}^T \psi(G_t) \leq \frac{1}{\sqrt{T}} \left(\Delta + \sqrt{r} \|\sigma_m\|_{F, \hat{\Gamma}} + \frac{1}{2} \|L_m\|_{\ell_1, \hat{\Gamma}^2} \right).$$

Theorem 4.15 provides a novel convergence guarantee for adaptive matrix-sign methods, revealing a controllable blockwise smoothness $\|L_m\|_{\ell_1, \hat{\Gamma}^2}$ in the final bound, mirroring the result of the vector case in Theorem 4.12.

If we remove the adaptive step size ($\Gamma_t = 1_{n \times m}$), the stationarity measure reduces to the nuclear norm $\psi(G_t) = \mathbb{E}[\|G_t\|_1]$, which is an upper bound on the nuclear norm of the true gradient, i.e., $\mathbb{E}[\|\nabla f(X_t)\|_1] \leq \mathbb{E}[\|G_t\|_1]$. This leads to the following result for non-adaptive spectral descent:

Corollary 4.16. Under the settings of Theorem 4.15, for $\sigma_m, L_m \in \mathbb{R}^{n \times m}$, if $\Gamma_t = 1_{n \times m}$ for all t , let $\eta =$

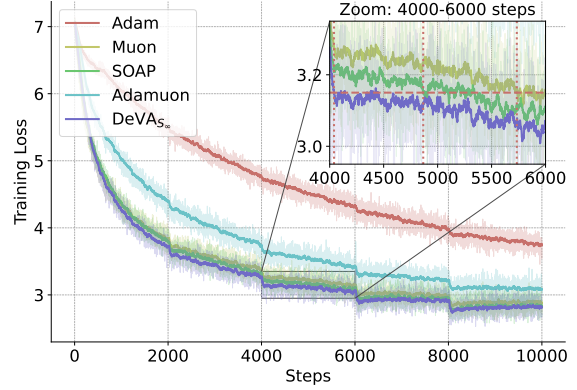


Figure 4. EMA-smoothed training loss for ViT-L/16 on ImageNet-1K during the first five epochs. All methods utilize a fixed learning rate of 0.001 without decay. DeVAs_∞ demonstrates significantly faster initial convergence compared to other optimizers.

$\sqrt{\frac{2\Delta}{T\|L_m\|_{\ell_1}}}$ and $n_t = \frac{r\|\sigma_m\|_F^2}{2\Delta\|L_m\|_{\ell_1}}T$, we have

$$\frac{1}{T} \sum_{i=1}^T \mathbb{E}[\|\nabla f(X_t)\|_1] \leq \frac{2}{\sqrt{T}} \left(\sqrt{2\Delta} \cdot \sqrt{\|L_m\|_{\ell_1}} \right).$$

This corollary shows that without variance adaptation, spectral descent achieves a batch complexity of $O(\Delta\|L_m\|_{\ell_1}\varepsilon^{-2})$ to reach ε -stationarity. Unlike $n_t = 1$ and a non-zero first-order momentum coefficient β_1 in (Shen et al., 2025), a large batch size $n_t = O(r\|\sigma_m\|_F^2\varepsilon^{-2})$ is required in our case and the total sample complexity will be $O(r\Delta\|\sigma_m\|_F^2\|L_m\|_{\ell_1}\varepsilon^{-4})$. This result matches the recent complexity established for Muon in (Shen et al., 2025, Corollary 4.2), while being more general in explaining how decoupled variance adaptation accelerates the convergence rate. The detailed proof is provided in Appendix D.2.

5. Experiment

Our implementation for computing the spectral-norm constrained update follows the Newton-Schultz iteration proposed in (Jordan et al., 2024b). In addition, the matrix sign function is multiplied by an RMS alignment in (Bai et al., 2025) for both Muon and DeVAs_∞ . All the following experiments are conducted on a single 40GB A100 GPU.

5.1. Main Results

Quadratic Trace Function Optimization. Neural network Hessians typically exhibit a block-diagonal structure (Ghorbani et al., 2019). Recent work (Zhang et al., 2024b; Orvieto & Gower, 2025) has utilized quadratic functions $f(x) = \frac{1}{2}x^T Hx$ with heterogeneous eigenspectra to illustrate why adaptive approaches are necessary in such settings. We extend this analysis to the matrix domain by considering the minimization of a quadratic trace function:

Table 2. Final validation perplexity and training runtime (ms/1k tokens) for a 270M NanoGPT model pretrained on 1.4B tokens. Values in ($\pm\%$) indicate relative gain or loss compared to Muon.

	Muon	SOAP	DeVA S_∞
Loss	3.305	3.304 (-0.0%)	3.271 (-1%)
Time	10.72s	15.79s (+47%)	16.49s (+54%)

$f(X) = \frac{1}{2} \text{Tr}(X^T H X)$ where $X, H \in \mathbb{R}^{d \times d}$ and H is a fixed matrix with a heterogeneous/homogeneous eigenspectrum. This simple optimization problem is of particular interest, as In-Context Learning (ICL) (Garg et al., 2022) of linear transformer can be framed as a special instance of the quadratic trace problem (Ma et al., 2026). Detailed notes for this example are provided in Appendix B.

As shown in Figure 2, deterministic gradient descent exhibits identical convergence rates across both homogeneous and heterogeneous functions. In contrast, the performance gain of DeVA S_∞ over the non-adaptive Muon mirrors the advantage of adaptive vector methods (DeVA ℓ_∞ , Adam) over the SignSGD. Notably, SOAP demonstrates similar convergence behaviors on both function types. We conjecture this occurs because SOAP lacks an explicit decoupling between the matrix sign update and the adaptive step size; consequently, it avoids the stagnant convergence patterns often observed in pure sign-based optimizers. Finally, Figure 2 reveals a significant reduction in the smoothness-related error $\|H\|_{1,r}$ when the function is Hessian-heterogeneous. This empirical reduction validates our theoretical intuition and underscores the importance of adaptive spectral methods for large-scale neural network training.

GPT. We evaluate our methods using the modded-nanogpt codebase (Jordan et al., 2024a), which incorporates modern architectural enhancements: rotary embeddings, RMSNorm, a linear decay schedule, and ReLU² activations. The snapshot we used includes three additional value embedding layers following the initial transformer blocks and an untied output head, bringing the total parameter count to 274M. We pretrain on the FineWeb-Edu dataset (Penedo et al., 2024); detailed hyperparameter sweeping experiments over learning rate and shampoo’s β are available in Appendix E. As shown in Figure 3 and Table 2, DeVA S_∞ consistently outperforms both Muon and SOAP in validation perplexity.

Image Classification. We evaluate performance on vision transformers (ViT-L/16 (Dosovitskiy, 2020)) using ImageNet-1K and convolutional neural networks (CNNs) on CIFAR-10. ImageNet-1K training trajectories are illustrated in Figure 4. As shown in Figure 5, following RMS alignment (Bai et al., 2025), the optimal learning rate consistently transfers across both vector and matrix optimizers, stabilizing between 0.005 and 0.01. For exhaustive experimental

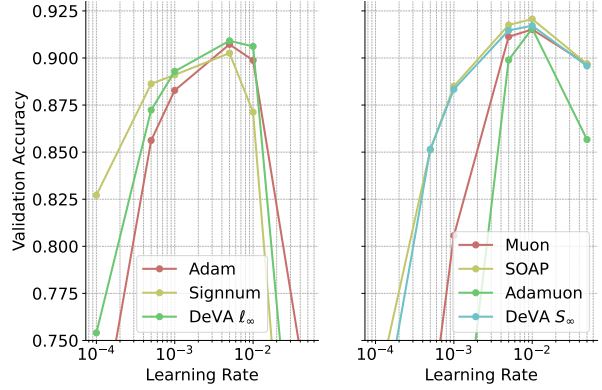


Figure 5. ResNet-20 validation accuracy on CIFAR-10 after 40 epochs. Optimal learning rates cluster near 0.005 for vector methods (left) and 0.01 for matrix methods (right), where DeVA ℓ_∞ and SOAP achieve peak performance, respectively.

configurations, refer to Appendices G and F.

Acknowledgements

We thank Yufan Huang for proofreading the manuscript. DFG would like to acknowledge DOE DE-SC0023162 Sparsitute MMICC center for partial support as well as NSF Non-linear graph IIS-2007481. ZZ would like to acknowledge support from N000142412621, Office of Naval Research. CSB and BB would like to acknowledge support from NSF IIS-2420724.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Amari, S.-I. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- An, K., Liu, Y., Pan, R., Ren, Y., Ma, S., Goldfarb, D., and Zhang, T. Asgo: Adaptive structured gradient optimization. *arXiv preprint arXiv:2503.20762*, 2025.
- Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Balles, L. and Hennig, P. Dissecting adam: The sign, magnitude and variance of stochastic gradients. In *International Conference on Machine Learning*, pp. 404–413. PMLR, 2018.

- Bernstein, J. and Newhouse, L. Modular duality in deep learning. *arXiv preprint arXiv:2410.21265*, 2024a.
- Bernstein, J. and Newhouse, L. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024b.
- Bernstein, J., Wang, Y.-X., Azizadenesheli, K., and Anandkumar, A. signsgd: Compressed optimisation for non-convex problems. In *International conference on machine learning*, pp. 560–569, 2018.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge University Press, 2004.
- Byrd, R. H., Hansen, S. L., Nocedal, J., and Singer, Y. A stochastic quasi-newton method for large-scale optimization. *SIAM Journal on Optimization*, 26(2):1008–1031, 2016.
- Candès, E. J. and Recht, B. Exact matrix completion via convex optimization. *Foundations of Computational Mathematics*, 9(6):717–772, April 2009. ISSN 1615-3383.
- Carlson, D., Cevher, V., and Carin, L. Stochastic spectral descent for restricted boltzmann machines. In *Artificial intelligence and statistics*, pp. 111–119. PMLR, 2015a.
- Carlson, D. E., Collins, E., Hsieh, Y.-P., Carin, L., and Cevher, V. Preconditioned spectral descent for deep learning. *Advances in neural information processing systems*, 28, 2015b.
- Chen, L., Liu, B., Liang, K., and Liu, Q. Lion secretly solves constrained optimization: As lyapunov predicts. *arXiv preprint arXiv:2310.05898*, 2023a.
- Chen, L., Li, J., and Liu, Q. Muon optimizes under spectral norm constraints. *arXiv preprint arXiv:2506.15054*, 2025.
- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., et al. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233, 2023b.
- Davis, D. and Drusvyatskiy, D. When do spectral gradient updates help in deep learning? *arXiv preprint arXiv:2512.04299*, 2025.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- DePavia, A., Cruzado, J., Liang, J., Charisopoulos, V., and Willett, R. Faster adaptive optimization via expected gradient outer product reparameterization. *arXiv preprint arXiv:2502.01594*, 2025.
- Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Duchi, J., Hazan, E., and Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Eschenhagen, R., Cai, A., Lee, T.-H., and Shi, H.-J. M. Clarifying shampoo: Adapting spectral descent to stochasticity and the parameter trajectory. *arXiv preprint arXiv:2602.09314*, 2026.
- Frangella, Z., Rathore, P., Zhao, S., and Udell, M. Promise: Preconditioned stochastic optimization methods by incorporating scalable curvature estimates. *Journal of Machine Learning Research*, 25(346):1–57, 2024a.
- Frangella, Z., Rathore, P., Zhao, S., and Udell, M. Sketchysgd: reliable stochastic optimization via randomized curvature estimates. *SIAM Journal on Mathematics of Data Science*, 6(4):1173–1204, 2024b.
- Frans, K., Abbeel, P., and Levine, S. What really matters in matrix-whitening optimizers? *arXiv preprint arXiv:2510.25000*, 2025a.
- Frans, K., Levine, S., and Abbeel, P. A stable whitening optimizer for efficient neural network training. *arXiv preprint arXiv:2506.07254*, 2025b.
- Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. What can transformers learn in-context? a case study of simple function classes. *Advances in neural information processing systems*, 35:30583–30598, 2022.
- Ghadimi, S. and Lan, G. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM journal on optimization*, 23(4):2341–2368, 2013.
- Ghorbani, B., Krishnan, S., and Xiao, Y. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning*, pp. 2232–2241. PMLR, 2019.
- Goemans, M. X. and Williamson, D. P. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, November 1995. ISSN 1557-735X. doi: 10.1145/227683.227684.
- Gonen, A., Orabona, F., and Shalev-Shwartz, S. Solving ridge regression using sketched preconditioned svrg. In

- International conference on machine learning*, pp. 1397–1405. PMLR, 2016.
- Gupta, V., Koren, T., and Singer, Y. Shampoo: Pre-conditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pp. 1842–1850. PMLR, 2018.
- Huang, Y., Seshadhri, C., and Gleich, D. F. Theoretical bounds on the network community profile from low-rank semi-definite programming. In *Proceedings of the 40th International Conference on Machine Learning, ICML 2023*, pp. 13976–13992, 2023.
- Jordan, K., Bernstein, J., Rappazzo, B., @fern-bear.bsky.social, Vlado, B., Jiacheng, Y., Cesista, F., Koszarsky, B., and @Grad62304977. modded-nanogpt: Speedrunning the nanogpt baseline, 2024a. URL <https://github.com/KellerJordan/modded-nanogpt/tree/ca964e982191830eebbd155e185937077511a8aa>.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024b. URL <https://kellerjordan.github.io/posts/muon/>.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Kulis, B., Surendran, A. C., and Platt, J. C. Fast low-rank semidefinite programming for embedding and clustering. In *Proceedings of the Eleventh International Conference on Artificial Intelligence and Statistics*, volume 2 of *Proceedings of Machine Learning Research*, pp. 235–242. PMLR, 2007.
- Large, T., Liu, Y., Huh, M., Bahng, H., Isola, P., and Bernstein, J. Scalable optimization in the modular norm. *Advances in Neural Information Processing Systems*, 37: 73501–73548, 2024.
- Li, Z., Liu, L., Liang, C., Chen, W., and Zhao, T. Normuon: Making muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025a.
- Liu, L., Xu, Z., Zhang, Z., Kang, H., Li, Z., Liang, C., Chen, W., and Zhao, T. Cosmos: A hybrid adaptive optimizer for memory-efficient training of llms. *arXiv preprint arXiv:2502.17410*, 2025b.
- Luo, X., Bai, C. S., Li, B., Drineas, P., Zhang, R., and Bullins, B. Stacey: Promoting stochastic steepest descent via accelerated lp-smooth nonconvex optimization. In *Forty-second International Conference on Machine Learning*, 2025.
- Ma, J., Huang, Y., Chi, Y., and Chen, Y. Preconditioning benefits of spectral orthogonalization in muon. *arXiv preprint arXiv:2601.13474*, 2026.
- Martens, J. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.
- Martens, J. and Grosse, R. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pp. 2408–2417. PMLR, 2015.
- Morwani, D., Shapira, I., Vyas, N., Malach, E., Kakade, S., and Janson, L. A new perspective on shampoo’s preconditioner. *arXiv preprint arXiv:2406.17748*, 2024.
- Nesterov, Y. et al. *Lectures on convex optimization*, volume 137. Springer, 2018.
- Orecchia, L. and Mahoney, M. W. Implementing regularization implicitly via approximate eigenvector computation. In Getoor, L. and Scheffer, T. (eds.), *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 121–128, New York, NY, USA, 2011.
- Orvieto, A. and Gower, R. In search of adam’s secret sauce. *arXiv preprint arXiv:2505.21829*, 2025.
- Penedo, G., Kydlíček, H., Lozhkov, A., Mitchell, M., Raffel, C. A., Von Werra, L., Wolf, T., et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37: 30811–30849, 2024.
- Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., and Cevher, V. Training deep learning models with norm-constrained lmos. *arXiv preprint arXiv:2502.07529*, 2025.
- Recht, B., Fazel, M., and Parrilo, P. A. Guaranteed minimum-rank solutions of linear matrix equations via nuclear norm minimization. *SIAM Review*, 52(3): 471–501, January 2010. ISSN 1095-7200. doi: 10.1137/070697835.
- Roosta-Khorasani, F. and Mahoney, M. W. Sub-sampled newton methods. *Mathematical Programming*, 174(1): 293–326, 2019.
- Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.

- Shen, W., Huang, R., Huang, M., Shen, C., and Zhang, J. On the convergence analysis of muon. *arXiv preprint arXiv:2505.23737*, 2025.
- Si, C., Zhang, D., and Shen, W. Adamuon: Adaptive muon optimizer. *arXiv preprint arXiv:2507.11005*, 2025.
- Tieleman, T. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26, 2012.
- Tong, T., Ma, C., and Chi, Y. Accelerating ill-conditioned low-rank matrix estimation via scaled gradient descent. *Journal of Machine Learning Research*, 22(150):1–63, 2021.
- Veprikov, A., Bolatov, A., Horváth, S., Beznosikov, A., Takáč, M., and Hanzely, S. Preconditioned norms: A unified framework for steepest descent, quasi-newton and adaptive methods. *arXiv preprint arXiv:2510.10777*, 2025.
- Vyas, N., Morwani, D., Zhao, R., Kwun, M., Shapira, I., Brandfonbrener, D., Janson, L., and Kakade, S. Soap: Improving and stabilizing shampoo using adam. *arXiv preprint arXiv:2409.11321*, 2024.
- Weinberger, K. and Saul, L. Unsupervised learning of image manifolds by semidefinite programming. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, volume 2, pp. 988–995. IEEE, 2004. doi: 10.1109/cvpr.2004.1315272.
- Wen, K., Hall, D., Ma, T., and Liang, P. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025.
- Wu, Y., Zhu, X., Wu, C., Wang, A., and Ge, R. Dissecting hessian: Understanding common structure of hessian in neural networks. *arXiv preprint arXiv:2010.04261*, 2020.
- Yao, Z., Gholami, A., Keutzer, K., and Mahoney, M. W. Pyhessian: Neural networks through the lens of the hessian. In *2020 IEEE international conference on big data (Big data)*, pp. 581–590. IEEE, 2020.
- Zhang, G., Fattahi, S., and Zhang, R. Y. Preconditioned gradient descent for overparameterized nonconvex burer–monteiro factorization with global optimality certification. *Journal of Machine Learning Research*, 24(163):1–55, 2023.
- Zhang, M., Liu, Y., and Schaeffer, H. Adagrad meets muon: Adaptive stepsizes for orthogonal updates. *arXiv preprint arXiv:2509.02981*, 2025.
- Zhang, Y., Chen, C., Ding, T., Li, Z., Sun, R., and Luo, Z. Why transformers need adam: A hessian perspective. *Advances in neural information processing systems*, 37: 131786–131823, 2024a.
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024b.

Appendix

Contents

A	Related Work	14
B	Trace Quadratic Example	16
C	Proofs for Section 3	17
C.1	Proof for Theorem 3.1	17
C.2	Proof for Proposition 3.2	18
D	Proofs for Section 4	18
D.1	Proof for Theorem 4.12	18
D.2	Proof for Theorem 4.15	19
E	Experiment Details on Modded NanoGPT	21
F	Experiment Details on ImageNet-1K	22
G	Experiment Details on CIFAR-10	23
H	Additional Results	24
H.1	Memory and runtime overhead.	24
H.2	Ablations	24
H.3	Second Moment Accumulation in DeVA_{S_∞}	25
H.4	Signal-to-noise ratio during pretraining	26

A. Related Work

In this section, we provide a structured overview of the optimization landscape relevant to our work, focusing on how different geometric perspectives influence convergence and efficiency in deep learning. We first review the history of adapting vector-based ideas to matrix variables in the domain of semi-definite programming. We next examine Gradient Descent under Non-Euclidean Norms, tracing the evolution from coordinate-wise sign methods to generalized ℓ_p geometries. We then discuss the emerging paradigm of Spectral Geometry and Modular Norms, which utilizes operator-norm updates to better align with the architectural properties of large-scale models. Finally, we survey Curvature-aware Methods, reviewing how local Hessian and Fisher information are efficiently approximated to navigate the complex loss surfaces of modern neural networks.

From vectors in linear programming to matrices in semi-definite programming Machine learning was a key driver of advanced semi-definite programming through the early 2000s. Semi-definite programming arises when a linear objective on a vector of variables is generalized to a linear objective on a matrix of variables, with the vector non-negativity constraints replaced by positive semi-definiteness of the matrix (Boyd & Vandenberghe, 2004). Many SDPs in machine learning arose as *convex* relaxations of combinatorial objectives, following the general paradigm exemplified by classical relaxations such as MaxCut (Goemans & Williamson, 1995). Famous examples of SDPs in ML include matrix completion with nuclear norms (Candès & Recht, 2009; Recht et al., 2010), which helped popularize the nuclear norm as the matrix-based analogue of the 1-norm in compressed sensing and sparse optimization. Related SDP formulations also appeared in manifold learning, embedding, and clustering problems (Weinberger & Saul, 2004; Kulis et al., 2007), motivated advanced algorithms for regularized graph diffusions (Orecchia & Mahoney, 2011), and more recently lower-bound on graph computations (Huang et al., 2023). A key modeling step in much of this research was adapting vector-based ideas to matrix variables, and our work studies descent algorithms within this framework.

Gradient Descent under non-Euclidean norm Starting from SignSGD (Bernstein et al., 2018), a significant line of research has studied sign-based methods, which can be viewed as (stochastic) steepest descent under the ℓ_∞ norm. This extends the classical ℓ_2 -based regime used in vanilla SGD (Ghadimi & Lan, 2013). By scaling down sparse noise and scaling up dense gradients comparatively, sign updates benefit learning when the gradients are dense but contain a sparse set of extremely noisy components (Bernstein et al., 2018).

Recent work on Lion (Chen et al., 2023b) and its generalization Lion- $\mathcal{K}$ (Chen et al., 2023a) further underscores the empirical benefits of sign-driven coordinates in large-scale tasks. More recently, Luo et al. (2025) interpolate between the extremes of non-Euclidean geometry (ℓ_2 and ℓ_∞) by considering general ℓ_p norms for $2 < p < \infty$, through which a more favorable Lipschitz constant L_p can be leveraged in high dimensions.

Spectral Geometry and Modular Norms Aside from ℓ_p norms, there is an increasing trend in research toward using spectral geometry for training deep learning models. In the Schatten- ∞ norm, the training loss of RBMs and CNNs admits a tighter majorization bound than in the Frobenius norm, motivating steepest descent in the spectral norm rather than the Euclidean norm (Carlson et al., 2015a;b).

Muon (Jordan et al., 2024b) applies a similar spectral update—implemented with a Newton–Schulz approximation to the polar factor—to the hidden layers of modern architectures and has shown strong empirical performance for NanoGPT-scale models and other large-scale training tasks (Wen et al., 2025; Liu et al., 2025a). The Scion optimizer (Pethick et al., 2025) builds a general framework of stochastic conditional gradient (LMO-based) methods with norm constraints; within this framework, Muon is a special case, while Scion allows for the use of operator norms (e.g., RMS, spectral, and ℓ_∞ norms) across different layers.

Building upon spectral and ℓ_∞ norms, the new concept of the modular norm (Bernstein & Newhouse, 2024a;b; Large et al., 2024) has been proposed to unify weights with different geometries in large-scale networks and popularize spectral and operator-norm updates. Recent research has also shown that spectral descent can outperform vanilla gradient descent when the matrix gradient has a high nuclear-to-Frobenius ratio (Shen et al., 2025; Davis & Drusvyatskiy, 2025), aligning with the dense gradient analysis for vector gradients in SignSGD (Bernstein et al., 2018).

Curvature-aware Methods In parallel, a variety of optimizers capture the intricate geometry of the loss surface by coupling parameter updates through local curvature information. Updates leveraging second-order information can achieve significantly more progress per iteration than simple scaled gradients. At the convex settings, preconditioned gradient

descent methods (Gonen et al., 2016; Frangella et al., 2024a) were among the first to utilize this geometric information. Subsequent Hessian estimation methods (Byrd et al., 2016; Roosta-Khorasani & Mahoney, 2019; Frangella et al., 2024b) maintain stochastic online approximations of the Hessian for neural networks. More recently, DePavia et al. (2025) showed that adaptive optimization methods can be accelerated via a change-of-basis technique named EGOP reparameterization that reduces the stable rank of a convex objective on deterministic optimization. For nonconvex matrix factorization problems, preconditioning has further emerged as a powerful accelerated optimization tool for both exactly and overly parameterized settings (Tong et al., 2021; Zhang et al., 2023). However, updates involving local curvature can be computationally prohibitive, as they typically require matrix inversions or matrix-vector products with the curvature matrix. Consequently, there is a strong demand for efficient methods that directly compute the inverse of high-quality curvature approximations.

Notable examples include K-FAC (Martens & Grosse, 2015) and Shampoo (Gupta et al., 2018), which approximate the Fisher information matrix via Kronecker products, and SOAP (Vyas et al., 2024), which approximates the Hessian using an optimal Kronecker product. To further mitigate the computational overhead, ASGO (An et al., 2025) only keeps a single-side preconditioner based on Shampoo’s preconditioner. To go beyond that, Adam (Kingma & Ba, 2014) maintains only the diagonal elements of the Gauss-Newton matrix.

To explain the success of such diagonal approximations, extensive prior work has shown that the Hessian of a neural network tends to exhibit a block-diagonal structure (Ghorbani et al., 2019; Zhang et al., 2024a;b), where each block corresponds to an individual layer or neuron. Furthermore, numerous studies (Wu et al., 2020; Yao et al., 2020) have observed that neural network Hessians are typically low-rank; their effective rank—measured by the ratio $\|H\|_*/\|H\|_{op}$ —is often significantly smaller than their ambient dimensionality.

To further accelerate spectral descent, particularly for optimizers like Muon, recent methods have begun incorporating second-order information. Optimizers such as Adamuon (Si et al., 2025), Normuon (Li et al., 2025), AdaGO (Zhang et al., 2025), and COSMOS (Liu et al., 2025b) integrate Adam-style adaptive stepsizes into the Muon update. Concurrent work (Eschenhagen et al., 2026) also shows that Shampoo achieves higher token efficiency than Muon, mirroring Adam’s advantage over Signum. However, the fundamental question regarding the theoretical origin and justification of second-order stepsizes within spectral gradient descent frameworks remains an open challenge.

B. Trace Quadratic Example

Our setup here is inspired directly from the results and discussions in (Zhang et al., 2024b; Orvieto & Gower, 2025). In particular, we consider the problem of minimizing the following function $f : \mathbb{R}^{d \times d} \rightarrow \mathbb{R}$:

$$f(X) = \frac{1}{2} \text{Tr}(X^T H X).$$

It is not too difficult to verify that $\nabla f(X) = \frac{1}{2}(H + H^T)X$ and $\nabla^2 f(X) = I_{d \times d} \otimes \frac{1}{2}(H + H^T)$. Thus, in order to construct a Heterogeneous (Homogeneous) Hessian for $f(X)$, we just need to construct a Heterogeneous (Homogeneous) H_{hom} and H_{het} .

Specifically, we fix the eigenvalues of H_{hom} and H_{het} , i.e., $\text{eig}(H_{hom}) = \text{eig}(H_{het}) = \{1, 2, 3, 99, 100, 101, 4998, 4999, 5000\}$. We choose both Hessians to be block-diagonal, with 3 by 3 blocks size. The homogeneous H_{hom} has eigenvalues of different magnitudes in each block, while the Heterogeneous H_{het} keeps similar magnitudes in each block, i.e.,

$$\begin{aligned} H_{hom} &= [[1, 2, 3], [99, 100, 101], [4998, 4999, 5000]] \\ H_{het} &= [[1, 99, 4998], [2, 100, 4999], [3, 101, 5000]]. \end{aligned}$$

To add randomness, we generate a 3 by 3 random rotation matrix $Q_{3 \times 3}$ to the block matrix of H_{het} and H_{hom} . Each rotation is sampled from the decomposition of a random 3 by 3 positive semidefinite matrix AA^T , where $A \in \mathbb{R}^{3 \times 3}$ has i.i.d. Gaussian entries.

One may notice that H_{hom} is indeed a permuted version of H_{het} , thus, running deterministic gradient descent on the two variants should show no discrepancy. However, under the stochastic case, the empirical learning process between the two variants demonstrated a significant difference (Zhang et al., 2024b; Orvieto & Gower, 2025). To introduce stochasticity in the gradient, we follow (Orvieto & Gower, 2025) to take the square root of the Hessian to define a 9 by 9 design A , i.e., $H = A^T A$. In practice, we use Cholesky decomposition for H to obtain A efficiently. Then the target function can be rewritten as

$$f(X) = \frac{1}{2} \text{Tr}(X^T A^T A X) = \frac{1}{2} \|AX\|_F^2.$$

Write row of A as a_i^T , then

$$f(X) = \frac{1}{2} \sum_{i=1}^d \|a_i^T X\|_2^2,$$

each row is like one ‘‘sample’’ and we can use the Kaczmarz randomized algorithm to sample the gradient of f .

In practice, all optimizers in Figure 2 except Gradient descent use moving average parameters β_1 set to zero and β_2 set to 0.99 to emphasize the effect of second-order adaptation. A learning rate scheduler with 50% warmup followed up by a linear decay to zero is implemented for optimizers except gradient descent.

C. Proofs for Section 3

C.1. Proof for Theorem 3.1

Theorem 3.1 (Coordinate-wise DeVAs_∞). *Let $L \in \mathbb{R}^{n \times n}$ and $R \in \mathbb{R}^{m \times m}$ be Kronecker factors with eigendecompositions $L = Q_L \Lambda_L Q_L^T$ and $R = Q_R \Lambda_R Q_R^T$. Let $\sigma_i = \sqrt{\lambda_i}$ and $\sigma_j = \sqrt{\mu_j}$ denote the singular values corresponding to the eigenvalues of L and R , respectively. With the Kronecker curvature approximation $\mathbb{E}[L^{1/2} \otimes R^{1/2}]$, and assuming local stability (see the proof), the optimal update ΔX^* in Equation (10) has spectral coordinate-wise form:*

$$\Delta X_{\text{DeVAs}_\infty}^* = -Q_L \left(\tilde{E}^{-1/2} \odot \text{msign}(Q_L^T G Q_R) \right) Q_R^T, \quad (14)$$

where $\tilde{E} \in \mathbb{R}^{n \times m}$ is the spectral adaptation matrix with entries defined by the ratio of expected to sample gradient singular value products:

$$\tilde{E}_{ij} = \frac{\mathbb{E}[\sigma_i \sigma_j]}{\sigma_i \sigma_j}. \quad (15)$$

Here, $\odot$ and the exponent $-1/2$ denote element-wise operations within the rotated spectral space.

Proof. Recall when $H = L^{1/2} \otimes R^{1/2}$, DeVA gives us

$$\text{vec}(\Delta X^*) = -\mathbb{E}[(L^{1/2} \otimes R^{1/2})]^{-1/2} (L^{1/2} \otimes R^{1/2})^{1/2} \cdot (L^{1/2} \otimes R^{1/2})^{-1/2} \text{vec}(G) \quad (20)$$

$$= -\mathbb{E}[(L \otimes R)^{1/2}]^{-1/2} (L \otimes R)^{1/4} \text{vec}(\text{msign}(G)) \quad (21)$$

Let $\mathcal{K} = \Lambda_L \otimes \Lambda_R$ be the diagonal matrix of Kronecker eigenvalues. Substituting the eigendecompositions $L = Q_L \Lambda_L Q_L^T$ and $R = Q_R \Lambda_R Q_R^T$ into the expectation, we have:

$$\mathbb{E}[L^{1/2} \otimes R^{1/2}] = (Q_L \otimes Q_R) \mathbb{E}[\mathcal{K}^{1/2}] (Q_L^T \otimes Q_R^T), \quad (22)$$

where we apply our local stability assumption that the eigenbases Q_L, Q_R are “locally stable” such that the expectation acts only on the eigenvalues. The vectorized update $\text{vec}(\Delta X^*)$ is given by:

$$\text{vec}(\Delta X^*) = -(Q_L \otimes Q_R) \mathbb{E}[\mathcal{K}^{1/2}]^{-1/2} (\mathcal{K}^{1/2})^{1/2} (Q_L^T \otimes Q_R^T) \text{vec}(\text{msign}(G)), \quad (23)$$

Using the property that for orthogonal Q , $\text{msign}(Q^T G Q) = Q^T \text{msign}(G) Q$, and applying the Kronecker identity $(A \otimes B) \text{vec}(X) = \text{vec}(A X B^T)$, we write:

$$\text{vec}(\Delta X^*) = -(Q_L \otimes Q_R) E^{-1/2} \text{vec}(\text{msign}(Q_L^T G Q_R)), \quad (24)$$

where $E = \mathbb{E}[\mathcal{K}^{1/2}] \mathcal{K}^{-1/2}$ is a diagonal matrix of size $nm \times nm$.

To simplify the operation, we observe that the k -th diagonal entry of E , corresponding to the index pair (i, j) , is:

$$E_{kk} = \frac{\mathbb{E}[\sqrt{\lambda_i \mu_j}]}{\sqrt{\lambda_i \mu_j}}. \quad (25)$$

Let $\sigma_i = \sqrt{\lambda_i}$ and $\sigma_j = \sqrt{\mu_j}$ denote the singular values of the gradient factors. We can represent the diagonal action of E^{-1} on the vectorized matrix as an element-wise product (Hadamard product) in matrix form. We define the spectral adaptation matrix $\tilde{E} \in \mathbb{R}^{n \times m}$ such that $\tilde{E}_{ij} = E_{kk}$. Specifically:

$$\tilde{E}_{ij} = \frac{\mathbb{E}[\sigma_i \sigma_j]}{\sigma_i \sigma_j}. \quad (26)$$

Substituting this back into the matrix form of the update rule and apply the identity $(A \otimes B) \text{vec}(X) = \text{vec}(A X B^T)$ again yields:

$$\Delta X^* = -Q_L \left(\tilde{E}^{-1/2} \odot (Q_L^T \text{msign}(G) Q_R) \right) Q_R^T. \quad (27)$$

This completes the proof. □

C.2. Proof for Proposition 3.2

Proposition 3.2. For a gradient $G \in \mathbb{R}^{n \times m}$, let $Q_L \Lambda_L Q_L^T$ and $Q_R \Lambda_R Q_R^T$ be the eigendecompositions of $L = GG^T$ and $R = G^T G$, respectively. Let λ_i and μ_j denote the i -th and j -th eigenvalues of L and R . If we define rotated gradient as $G' = Q_L^T G Q_R$, then:

$$\lambda_i = e_i^T (G' \odot G') 1_m, \quad \mu_j = 1_n^T (G' \odot G') e_j. \quad (16)$$

Proof. Let $Q_L = [u_1, \dots, u_n]$ and $Q_R = [v_1, \dots, v_m]$. The rotated gradient is given by $G' = Q_L^T G Q_R$, with entries $G'_{ij} = u_i^T G v_j$. Thus $e_i^T (G' \odot G') 1_m = \sum_{k=1}^m (G'_{ik})^2 = \sum_{k=1}^m (u_i^T G Q_R e_k)^2 = \|u_i^T G Q_R\|_2^2 = u_i^T G G^T u_i = \lambda_i$. The derivation for μ_j follows symmetrically. $\square$

D. Proofs for Section 4

Setup and notation. Let $\{\xi_t\}_{t \geq 1}$ denote the sequence of i.i.d. mini-batch indices drawn during training with size of n_t . Define the natural filtration

$$\mathcal{F}_0 := \{\emptyset, \Omega\}, \quad \mathcal{F}_t := \sigma(\xi_1, \xi_2, \dots, \xi_t) \quad \text{for } t \geq 1.$$

D.1. Proof for Theorem 4.12

Theorem 4.12. Under Assumption 4.8 and Assumption 4.10, for $\sigma_v, L_v, \gamma_t \in \mathbb{R}^d$, let $\Delta = f(x_1) - f^*$, for a fixed learning rate $\eta = 1/\sqrt{T}$, a mini-batch size $n_t = T$, suppose the weight process $\{\gamma_t\}_{t \geq 0}^d$ is bounded and $\mathcal{F}_{t-1}$ measurable, let $\hat{\gamma} = \frac{1}{T} \sum_{t=1}^T \gamma_t$ and $\hat{\gamma}^2 = \frac{1}{T} \sum_{t=1}^T \gamma_t^2$, the iterates using Equation (18) satisfy:

$$\frac{1}{T} \sum_{t=1}^T \|\nabla f(x_t)\|_{1, \gamma_t} \leq \frac{1}{\sqrt{T}} \left(\Delta + 2\|\sigma_v\|_{1, \hat{\gamma}} + \frac{1}{2}\|L_v\|_{1, \hat{\gamma}^2} \right).$$

Proof. Using blockwise smooth assumption in Assumption 4.8 and broadcast $L_{v,i}$ in each block to form a $\mathbb{R}^d$ vector yields the following descent inequality

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) + \langle \nabla f(x_t), x_{t+1} - x_t \rangle + \frac{1}{2} \sum_{i=1}^c L_{v,i} \|x_{t+1,i} - x_{t,i}\|_2^2 \\ &\leq f(x_t) - \eta \sum_{i=1}^d \gamma_{t,i} \nabla_i f(x_t) \text{sign}(g_t)_i + \frac{\eta^2}{2} \sum_{i=1}^c L_{v,i} \sum_{j \in \mathcal{B}_i} \gamma_{t,j}^2. \end{aligned}$$

Since we broadcast $L_{v,i}$ for all $j \in \mathcal{B}_i$, we can push $L_{v,i}$ inside the inner sum, then the above inequality equivalent to

$$f(x_{t+1}) \leq f(x_t) - \eta \sum_{i=1}^d \gamma_{t,i} \nabla_i f(x_t) \text{sign}(g_t)_i + \frac{\eta^2}{2} \sum_{i=1}^d L_{v,i} \gamma_{t,i}^2,$$

using the sign mismatching trick used in (Bernstein et al., 2018), we know $\nabla_i f(x_t) \text{sign}(g_t)_i = |\nabla_i f(x_t)| - 2|\nabla_i f(x_t)|\mathbb{I}[\text{sign}(\nabla_i f(x_t)) \neq \text{sign}(g_t)_i]$, plug this back into the above descent inequality yields:

$$f(x_{t+1}) \leq f(x_t) - \eta \sum_{i=1}^d |\nabla f(x_t)|_i \gamma_{t,i} + 2\eta \sum_{i=1}^d \gamma_{t,i} |\nabla f(x_t)|_i \mathbb{I}[\text{sign}(\nabla_i f(x_t)) \neq \text{sign}(g_t)_i] + \frac{\eta^2}{2} \sum_{i=1}^d L_{v,i} \gamma_{t,i}^2,$$

Conditioned on $\mathcal{F}_{t-1}$, we take the expectation over ξ_t on both sides (we write $\mathbb{E}_{\xi_t}[\cdot | \mathcal{F}_{t-1}]$ as $\mathbb{E}[\cdot]$ for short), and use the

property of $\Pr[\text{sign}(\nabla_i f(x_t)) \neq \text{sign}(g_t)_i] \leq \frac{\sigma_{v,t,i}}{|\nabla f(x_t)|_i}$, this yield

$$\begin{aligned} f(x_{t+1}) &\leq \mathbb{E}[f(x_t)] - \eta \sum_{i=1}^d |\nabla f(x_t)|_i \gamma_{t,i} + 2\eta \sum_{i=1}^d \gamma_{t,i} \sigma_{v,t,i} + \frac{\eta^2}{2} \sum_{i=1}^d L_{v,i} \gamma_{t,i}^2 \\ &\leq \mathbb{E}[f(x_t)] - \eta \sum_{i=1}^d |\nabla f(x_t)|_i \gamma_{t,i} + \frac{2\eta}{\sqrt{T}} \sum_{i=1}^d \gamma_{t,i} \sigma_{v,i} + \frac{\eta^2}{2} \sum_{i=1}^d L_{v,i} \gamma_{t,i}^2 \\ &\leq \mathbb{E}[f(x_t)] - \eta \|\nabla f(x_t)\|_{1,\gamma_t} + \frac{2\eta}{\sqrt{T}} \|\sigma_v\|_{1,\gamma_t} + \frac{\eta^2}{2} \|L_v\|_{1,\gamma_t^2}, \end{aligned}$$

where the second inequality comes from $\sigma_{v,t,i}^2 \leq \sigma_{v,i}/T$ when $n_t = T$ and the final inequality comes from the definition of γ -weighted norms. We then take the expectation over all previous $\xi_t, t \leq T$, and take the telescope sum up to T gives us

$$\begin{aligned} \sum_{t=1}^T \eta \|f(x_t)\|_{1,\gamma_t} &\leq f(x_1) - \mathbb{E}[f(x_{T+1})] + \frac{2\eta}{\sqrt{T}} \sum_{t=1}^T \|\sigma_v\|_{1,\gamma_t} + \frac{\eta^2}{2} \sum_{t=1}^T \|L_v\|_{1,\gamma_t^2}, \\ \frac{1}{T} \sum_{t=1}^T \|f(x_t)\|_{1,\gamma_t} &\leq \frac{f(x_1) - \mathbb{E}[f(x_{T+1})]}{\eta T} + \frac{2}{\sqrt{T}} \|\sigma_v\|_{1,\hat{\gamma}} + \frac{\eta}{2} \|L_v\|_{1,\hat{\gamma}^2}, \end{aligned}$$

If we let $\Delta = f(x_1) - \min f(x)$ and $\eta = \frac{1}{\sqrt{T}}$, we have

$$\frac{1}{T} \sum_{t=1}^T \|f(x_t)\|_{1,\gamma_t} \leq \frac{1}{\sqrt{T}} \left(\Delta + 2\|\sigma_v\|_{1,\hat{\gamma}} + \frac{1}{2} \|L_v\|_{1,\hat{\gamma}^2} \right).$$

□

D.2. Proof for Theorem 4.15

Theorem 4.15. Under Assumption 4.13 and Assumption 4.10, for $\sigma_m, L_m, \Gamma_t \in \mathbb{R}^{n \times m}$, let $r = \min(n, m)$ and $\Delta = f(X_1) - f^*$, for a learning rate $\eta = 1/\sqrt{T}$ and mini-batch size $n_t = T$, suppose the weight process $\{\Gamma_t\} \subset \mathbb{R}_{>0}^{n \times m}$ is bounded and $\mathcal{F}_{t-1}$ measurable, let $\hat{\Gamma} = \sqrt{\frac{1}{T} \sum_{t=1}^T \Gamma_t^2}$ and $\hat{\Gamma}^2 = \frac{1}{T} \sum_{t=1}^T \Gamma_t^2$. Defining the stationarity measure $\psi(G_t) = \mathbb{E}[\langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t}]$, the iterates using Equation (19) satisfy:

$$\frac{1}{T} \sum_{i=1}^T \psi(G_t) \leq \frac{1}{\sqrt{T}} \left(\Delta + \sqrt{r} \|\sigma_m\|_{F,\hat{\Gamma}} + \frac{1}{2} \|L_m\|_{\ell_1,\hat{\Gamma}^2} \right).$$

Proof. The L -smooth assumption in Assumption 4.13 yields the following descent inequality:

$$\begin{aligned} f(X_{t+1}) &\leq f(X_t) + \langle \nabla f(X_t), X_{t+1} - X_t \rangle + \frac{1}{2} \|\sqrt{L_m} \odot (X_{t+1} - X_t)\|_F^2 \\ &= f(X_t) + \langle \nabla f(X_t), -\eta \Gamma_t \odot \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \|\sqrt{L_m} \odot \Gamma_t \odot \text{msign}(G_t)\|_F^2, \end{aligned}$$

using $\|A \odot B\|_F \leq \|A\|_F \|B\|_F$, $\langle A \odot B, C \rangle = \langle B, A \odot C \rangle$, and $|\text{msign}(G_t)_{ij}| = |\sum_{k=1}^r U_{ik} V_{jk}| \leq \|U_{i,:}\|_2 \cdot \|V_{j,:}\|_2 \leq 1$, we have,

$$f(X_{t+1}) \leq f(X_t) + \langle \Gamma_t \odot \nabla f(X_t), -\eta \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \sum_{ij} L_{m,ij} \Gamma_{t,ij}$$

observe that, in ℓ_1 norm, we have $\|\Gamma_t \odot \Gamma_t^2\|_{\ell_1} = \|L_m\|_{\ell_1, \Gamma_t^2}$, this yields

$$f(X_{t+1}) \leq f(X_t) + \langle \Gamma_t \odot \nabla f(X_t), -\eta \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2}.$$

now we let $\nabla f(X_t) = G_t - E_t$. E_t is the error matrix and $\mathbb{E}_{\xi_t}[E_t|\mathcal{F}_{t-1}] = 0$ by the unbiased gradient assumption, plug this into the above equation, we have

$$\begin{aligned}
 f(X_{t+1}) &\leq f(X_t) + \langle \Gamma_t \odot (G_t - E_t), -\eta \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2} \\
 &\leq f(X_t) + \langle \Gamma_t \odot G_t, -\eta \text{msign}(G_t) \rangle + \langle \Gamma_t \odot E_t, \eta \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2} \\
 &\leq f(X_t) - \eta \langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t} + \langle \Gamma_t \odot E_t, \eta \text{msign}(G_t) \rangle + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2} \\
 &\leq f(X_t) - \eta \langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t} + \eta \|\Gamma_t \odot E_t\|_1 + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2},
 \end{aligned}$$

where the final inequality comes from the Holder inequality. Condition on $\mathcal{F}_{t-1}$, now we take the expectation over ξ_t on both sides (we write $\mathbb{E}_{\xi_t}[\cdot|\mathcal{F}_{t-1}]$ as $\mathbb{E}[\cdot]$ for short), this yields

$$\begin{aligned}
 f(X_{t+1}) &\leq \mathbb{E}[f(X_t)] - \eta \mathbb{E}[\langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t}] + \eta \mathbb{E}[\|\Gamma_t \odot E_t\|_1] + \frac{\eta^2}{2} \mathbb{E}[\|L_m\|_{\ell, \Gamma_t^2}] \\
 &= \mathbb{E}[f(X_t)] - \eta \mathbb{E}[\langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t}] + \eta \mathbb{E}[\|\Gamma_t \odot E_t\|_1] + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2}
 \end{aligned}$$

where the final equality uses $\{\Gamma_t\}$ is $\mathcal{F}_{t-1}$ measurable. Now we want to study the error term $\mathbb{E}[\|\Gamma_t \odot E_t\|_1]$, we know the nuclear norm is upper bounded by the Frobenius norm times the square root of the rank of the matrix, now we have

$$\mathbb{E}[\|\Gamma_t \odot E_t\|_1] \leq \sqrt{r} \mathbb{E}[\|\Gamma_t \odot E_t\|_F] \leq \sqrt{r} \sqrt{\mathbb{E}[\|\Gamma_t \odot E_t\|_F^2]} = \sqrt{r} \sqrt{\mathbb{E}[\sum_{i=1}^n \sum_{j=1}^m \Gamma_{t,ij}^2 E_{t,ij}^2]},$$

where is second inequality comes from the concavity of $\sqrt{X}$. Under the linearity of expectation and $\mathcal{F}_{t-1}$ -measurability of Γ_t , we have $\mathbb{E}[\sum_{i=1}^n \sum_{j=1}^m \Gamma_{t,ij}^2 E_{t,ij}^2] = \sum_{i=1}^n \sum_{j=1}^m \Gamma_{t,ij}^2 \sigma_{t,ij}^2$ (Here we simplify σ_m to σ for brevity). When $n_t = T$, we further have $\sigma_{t,ij}^2 \leq \sigma_{ij}^2/T$ and this gives us $\mathbb{E}[\|\Gamma_t \odot E_t\|_1] \leq \sqrt{\frac{r}{T}} \sqrt{\sum_{i=1}^n \sum_{j=1}^m \Gamma_{t,ij}^2 \sigma_{ij}^2} = \sqrt{\frac{r}{T}} \|\sigma_m\|_{F, \Gamma_t}$. Plug this back into the previous descent inequality, and it yields

$$f(X_{t+1}) \leq \mathbb{E}[f(X_t)] - \eta \mathbb{E}[\langle G_t, \text{msign}(G_t) \rangle_{\Gamma_t}] + \eta \sqrt{\frac{r}{T}} \|\sigma_m\|_{F, \Gamma_t} + \frac{\eta^2}{2} \|L_m\|_{\ell, \Gamma_t^2}.$$

Take the expectation over previous $\xi_t, t \leq T$ and apply the tower rule, and take the telescope sum of the above descent inequality up to T , which gives us

$$\begin{aligned}
 \sum_{t=1}^T \eta \psi(G_t) &\leq f(X_1) - \mathbb{E}[f(X_{T+1})] + \eta \sqrt{\frac{r}{T}} \sum_{t=1}^T \|\sigma_m\|_{F, \Gamma_t} + \frac{\eta^2}{2} \sum_{t=1}^T \|L_m\|_{\ell, \Gamma_t^2} \\
 \frac{1}{T} \sum_{t=1}^T \psi(G_t) &\leq \frac{f(X_1) - \mathbb{E}[f(X_{T+1})]}{\eta T} + \sqrt{\frac{r}{T}} \|\sigma_m\|_{F, \hat{\Gamma}} + \frac{\eta}{2} \|L_m\|_{\ell, \hat{\Gamma}^2},
 \end{aligned}$$

If we let $\Delta = f(X_1) - \min f(X)$ and $\eta = \frac{1}{\sqrt{T}}$, we have

$$\frac{1}{T} \sum_{t=1}^T \psi(G_t) \leq \frac{1}{\sqrt{T}} \left(\Delta + \sqrt{r} \|\sigma_m\|_{F, \hat{\Gamma}} + \frac{1}{2} \|L_m\|_{\ell, \hat{\Gamma}^2} \right).$$

□

E. Experiment Details on Modded NanoGPT

Experiment Setup. In our NanoGPT experiments, we utilized the May 27 snapshot of modded-nanogpt (Jordan et al., 2024a), which incorporates several modern architectural enhancements: rotary embeddings, RMSNorm, a linear decay scheduler, and ReLU² activations. This version also features an untied output head and three additional value embedding layers appended to the early transformer layer outputs. Each embedding block consists of approximately 38M parameters ($768 \times 50, 257$); these four blocks total 152M parameters, resulting in a model size of 276M parameters. We train our model on the FineWeb-10B dataset (Penedo et al., 2024). To stabilize training, we incorporate a sign-trick similar to that proposed in (Si et al., 2025) within the msign function. We employ a warmup phase spanning 60% of the total steps (at a fixed learning rate) for the linear decay schedule. The optimizer hyperparameters in Figure 1 and Figure 3 are detailed in Table 3.

Table 3. Hyperparameter configurations for the various optimizers evaluated in Figure 1 and Figure 3. For methods employing Nesterov momentum, the acceleration term is calculated as $\text{sign}(\beta_1 m_t + (1 - \beta_1)g_t)$ or $\text{msign}(\beta_1 m_t + (1 - \beta_1)g_t)$, where m_t is the moving average and g_t is the current gradient.

Hyperparameter	Adam	Lion	MUON	SOAP	Adamuon	DeVA _{S_∞}
Learning Rate (η)	0.001	0.001	0.001	0.001	0.001	0.001
Weight Decay	0.1	0.1	0.1	0.1	0.1	0.1
Batch Size (tokens)	49152	49152	49152	49152	49152	49152
β_1 (First Moment)	0.9	0.95	0.95	0.95	0.95	0.95
β_2 (Second Moment)	0.99	0.95	0.95	0.95	0.95	0.95
β_3 (Shampoo Moment)	–	–	0.95	0.95	–	0.95
ε (Stability)	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}
eigdecomp Frequency	–	–	–	10	–	10
Nesterov Acceleration	False	True	False	False	False	False

Hyperparameter Robustness. We further evaluate the hyperparameter robustness of DeVA_{S_∞} with respect to the learning rate and β_3 in Figure 6, Table 4, and Table 5. As shown in the learning-rate sweep results in Figure 6 and Table 4, DeVA_{S_∞} outperforms both SOAP and Muon when the learning rate is set to 10^{-3} . Importantly, we also find that DeVA_{S_∞} is robust to the choice of β_3 : all tested values of β_3 achieve validation losses lower than the best result obtained by SOAP (3.2769).

Table 4. Numerical learning-rate sweep (1.4B tokens).

Optimizer	LR = 10^{-4}	LR = 5×10^{-4}	LR = 10^{-3}	LR = 5×10^{-3}	LR = 10^{-2}
Muon	3.2819	3.2804	3.3038	3.3803	3.4015
SOAP	3.3686	3.2769	3.2808	3.3313	3.4422
DeVA _{S_∞}	3.3450	3.2729	3.2709	3.3382	3.4174

Table 5. **Robustness of DeVA_{S_∞} to β_3 .** Validation loss when varying β_3 with the learning rate fixed to 10^{-3} (1.4B-token setting; same evaluation as above). The bold entry is the lowest loss in this β_3 grid and corresponds to the configuration we emphasize in the rebuttal.

β_3	0.8	0.9	0.95	0.975	0.9875
Validation loss	3.2719	3.2687	3.2743	3.2749	3.2723

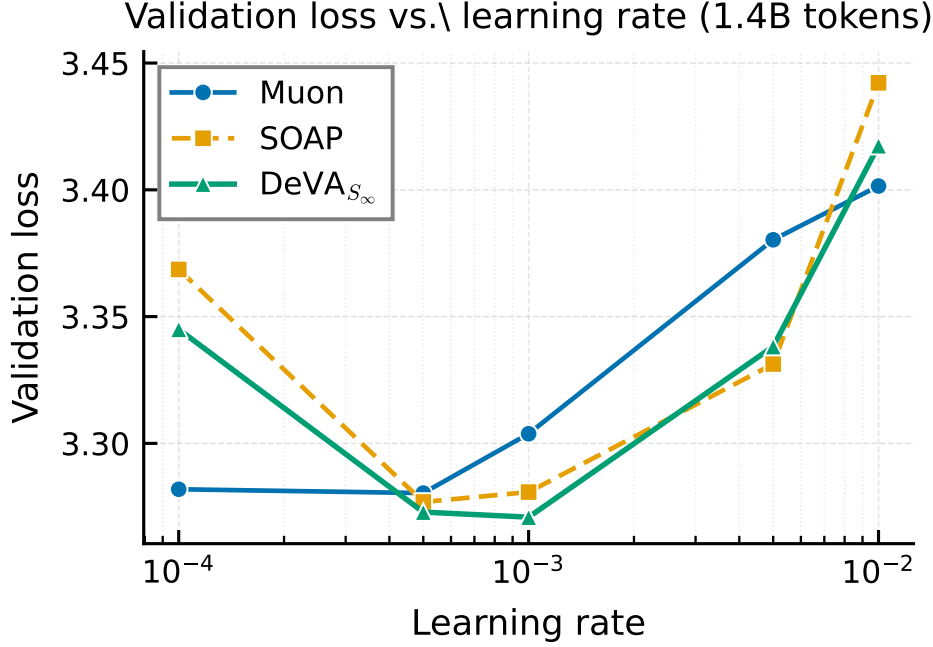


Figure 6. **Learning-rate sensitivity (1.4B tokens).** Validation loss versus learning rate for Muon, SOAP, and DeVA_{S_∞} under a fixed token budget.

F. Experiment Details on ImageNet-1K

Experiment Setup. For our ImageNet experiments, we train a transformer model ViT-L/16 on the ImageNet-1K dataset from ILSVRC2012 (Deng et al., 2009). We compare convergence behavior using early stopping within a five-epoch window and no learning rate decay. To stabilize training, we incorporate a sign-trick similar to that proposed in (Si et al., 2025) within the msign function. We observe that a sign-stabilized msign update is beneficial for image classification tasks. The hyperparameter configurations corresponding to Figure 4 are detailed in Table 6.

Table 6. Hyperparameter configurations for the various optimizers evaluated in Figure 4. For methods employing Nesterov momentum, the acceleration term is calculated as $\text{sign}(\beta_1 m_t + (1 - \beta_1)g_t)$ or $\text{msign}(\beta_1 m_t + (1 - \beta_1)g_t)$, where m_t is the moving average and g_t is the current gradient.

Hyperparameter	Adam	MUON	SOAP	Adamuon	DeVA _{S_∞}
Learning Rate (η)	0.001	0.001	0.001	0.001	0.001
Weight Decay	0.1	0.1	0.1	0.1	0.1
Batch Size (imgs)	512	512	512	512	512
β_1 (First Moment)	0.9	0.95	0.95	0.95	0.95
β_2 (Second Moment)	0.999	0.95	0.95	0.95	0.95
β_3 (Shampoo Moment)	–	–	0.95	–	0.95
ε (Stability)	10 ⁻⁸	10 ⁻⁸	10 ⁻⁸	10 ⁻⁸	10 ⁻⁸
eigdecomp Frequency	–	–	10	–	10
Nesterov Acceleration	False	True	False	False	True

G. Experiment Details on CIFAR-10

Experiment Setup. For our CIFAR-10 experiments, we compare the validation accuracy of ResNet-20 over 40 epochs using a linear learning rate decay without any warm-ups. The hyperparameter configurations for each optimizer—selected based on peak validation accuracy in Figure 5—are provided in Table 7.

Table 7. Best Hyperparameter settings across different optimizers on ResNet-20 in Figure 5. For methods employing Nesterov momentum, the acceleration term is calculated as $\text{sign}(\beta_1 m_t + (1 - \beta_1)g_t)$ or $\text{msign}(\beta_1 m_t + (1 - \beta_1)g_t)$, where m_t is the moving average and g_t is the current gradient.

Hyperparameter	Adam	Signum	DeVA $_{\ell_\infty}$	MUON	SOAP	Adamuon	DeVA $_{S_\infty}$
Learning rate (η)	0.05	0.05	0.05	0.001	0.001	0.001	0.001
Weight Decay	0.1	0.1	0.1	0.1	0.1	0.1	0.1
Batch Size (imgs)	128	128	128	128	128	128	128
β_1 (First Moment)	0.9	0.9	0.9	0.95	0.95	0.95	0.95
β_2 (Second Moment)	0.999	0.999	0.999	0.95	0.95	0.95	0.95
β_3 (Shampoo Moment)	—	—	—	—	0.95	—	0.95
ε (Stability)	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}	10^{-8}
eigdecomp Frequency	—	—	—	—	5	—	5
Nesterov Acceleration	False	False	False	False	False	False	False

H. Additional Results

We summarize the per-step computational and memory costs of the matrix optimizers Muon, SOAP, and DeVA_{S_∞} in Table 8 of Appendix H.1. The main runtime bottleneck of DeVA_{S_∞} is the additional eigendecomposition step, while the memory bottleneck comes from storing the covariance matrix and second-order moment matrices. In Appendix H.2, we discuss how to reduce the runtime complexity by increasing the batch size and decreasing the eigendecomposition frequency. In Appendix H.3, we investigate whether a reduced second-order moment estimator can achieve performance comparable to the original DeVA_{S_∞} . Finally, in Appendix H.4, we present the evolution of the signal-to-noise ratio (SNR) across different layers in Table 10, observing that the SNR peaks during the early stage of training and remains stable as training progresses.

H.1. Memory and runtime overhead.

Table 8. Asymptotic per-step cost (matrix shape $n \times m$). Memory footprint and floating-point work per optimizer step, using the same symbols and conventions as the main text. The symbol freq denotes how often expensive subroutines (for example, eigendecompositions) are invoked relative to the baseline step; these terms appear explicitly so that amortization over an inner loop is not hidden. The last column identifies the dominant subroutine for each method. This table is included to make the complexity discussion in our response self-contained.

Optimizer	Memory	FLOPs per step	Dominant subroutine
Muon	$\mathcal{O}(5nm^2)$	$\mathcal{O}(3nm)$	Newton–Schulz iteration
SOAP	$\mathcal{O}\left(n^2m + \frac{n^3+m^3}{\text{freq}}\right)$	$\mathcal{O}(n^2 + m^2 + 4nm)$	Eigendecomposition; covariance
DeVA_{S_∞}	$\mathcal{O}\left(n^2m + 5nm^2 + \frac{n^3+m^3}{\text{freq}}\right)$	$\mathcal{O}(n^2 + m^2 + 4nm)$	Eigendecomposition; covariance; N–S

H.2. Ablations

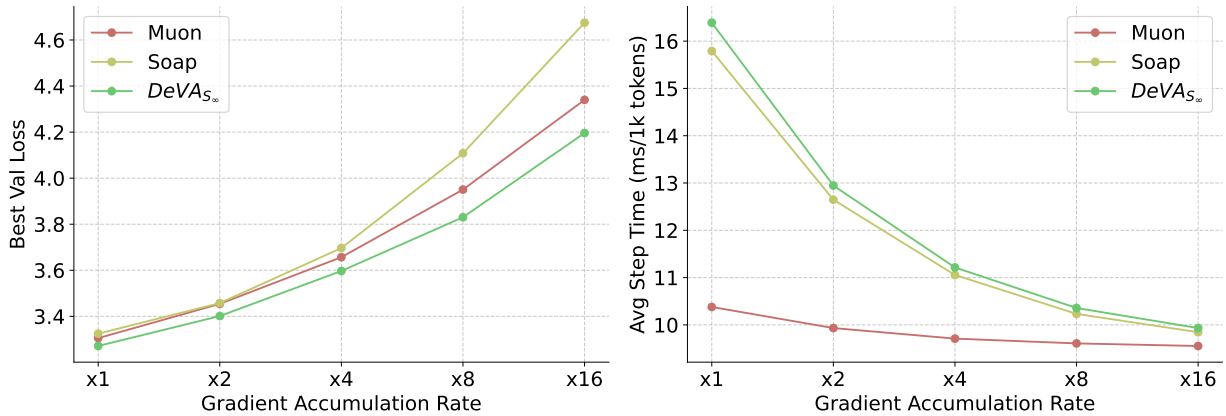


Figure 7. Batch size sensitivity on NanoGPT (274M). The generalization performance of DeVA_{S_∞} remains robust across larger batch sizes. Furthermore, the performance gap narrows the difference in average runtime between Muon and DeVA_{S_∞} at higher batch scales.

Different batch sizes We evaluate the impact of batch size by varying the number of gradient accumulation steps. As shown in Figure 7, validation loss increases across all optimizers as the gradient accumulation rate rises; notably, SOAP exhibits the most significant performance degradation. Most importantly, DeVA_{S_∞} consistently achieves the lowest validation loss across all accumulation settings. Muon performs competitively, generally maintaining a performance profile between DeVA_{S_∞} and SOAP.

Regarding computational efficiency, Muon remains the fastest optimizer in terms of average step time across all accumulation levels. For all methods, average step time decreases as the accumulation rate increases, likely due to the amortization of optimizer overhead (such as `eigdecomp` and `msign`) over a larger number of micro-batches. While DeVA_{S_∞} and SOAP incur higher overhead than Muon at low accumulation rates (e.g., rate = 1), this gap narrows significantly as the accumulation rate increases.

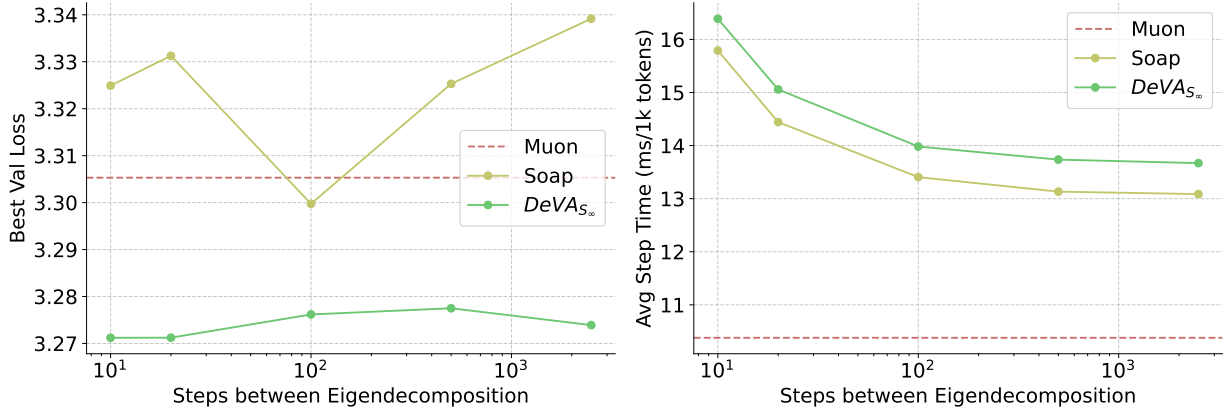


Figure 8. Sensitivity to eigen-decomposition frequency on NanoGPT (274M). DeVA_{S_∞} maintains consistently low validation loss even as the frequency of eigen-decomposition is reduced, demonstrating its robustness to stale updates.

Robustness to eigen-decomposition frequency. As illustrated in Figure 8, DeVA_{S_∞} exhibits remarkable stability; its validation loss remains consistently lower than that of SOAP, fluctuating only minimally (around 3.27) even as the eigen-decomposition frequency is significantly reduced. While SOAP maintains a slightly higher validation loss (between 3.30 and 3.34), it also demonstrates relative resilience to less frequent updates.

There is a pronounced efficiency gain as the eigen-decomposition interval increases. Both optimizers experience a sharp reduction in average step time when moving from a frequency of 10 to 100 steps, with diminishing returns for higher intervals. This confirms that reducing the frequency of the eigen-decomposition step significantly lowers computational overhead without substantially compromising model convergence.

H.3. Second Moment Accumulation in DeVA_{S_∞}

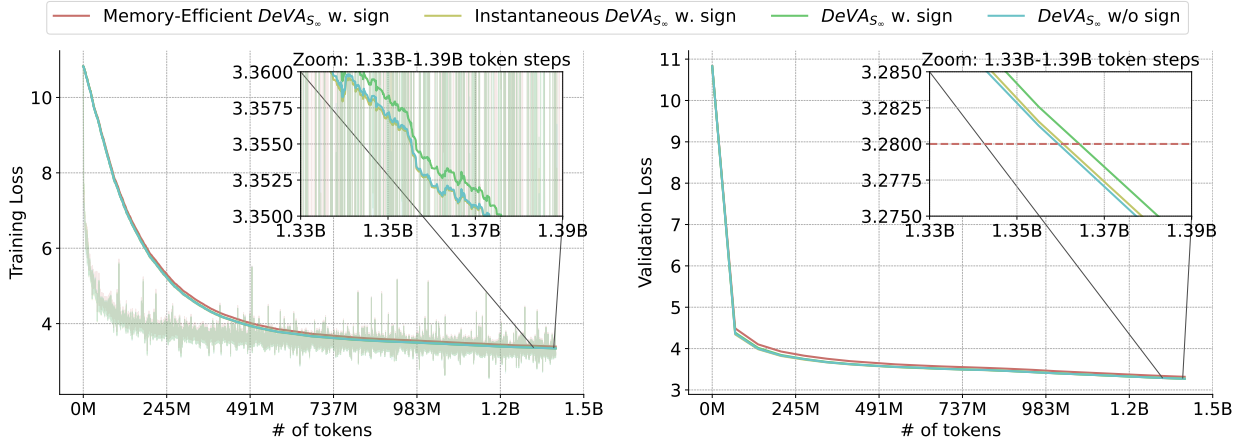


Figure 9. Training and Validation Performance of different DeVA_{S_∞} variants on NanoGPT (274M).

In this study, we evaluate two variants of DeVA_{S_∞} distinguished by their second-moment accumulation strategies:

- **Variant (a) (Memory-Efficient DeVA_{S_∞} , Algorithm 3):** Following the approach in Adafactor (Shazeer & Stern, 2018), which achieves sublinear memory complexity by separately accumulating the squared row and column sums of the gradient G_t , we develop a memory-efficient second-order moment for DeVA_{S_∞} . By independently accumulating row sums r_t and column sums c_t (Lines 14–15, Algorithm 3), we reduce the memory overhead of the second-order moments from $O(nm)$ to $O(n + m)$.
- **Variant (b) (Instantaneous DeVA_{S_∞} , Algorithm 4):** In contrast to calculating the second-order moments r_t and c_t by EMA of G'_t , we introduce an Adam-style variant that calculates them based on the sample gradient G'_t (Lines 12–13).

Beyond these accumulation strategies, we also explore the impact of the sign-stabilization trick within the `msign` function. While previously shown to benefit ImageNet classification, we evaluate its general utility on language modeling. We append the suffixes *w. sign* and *w/o sign* to DeVA_{S_∞} to denote whether the `msign` operator includes this stabilization.

Table 9. Best Validation loss across DeVA_{S_∞} variants on NanoGPT (274M).

	DeVA_{S_∞} <i>w. sign</i>	DeVA_{S_∞} <i>w/o sign</i>	Eff. DeVA_{S_∞} <i>w. sign</i>	Inst. DeVA_{S_∞} <i>w. sign</i>
Best Val Loss	3.271	3.269	3.309	3.270

The training and validation curves are illustrated in Figure 9, with detailed validation results summarized in Table 9. Our results demonstrate that employing sublinear memory for second-order momentum incurs only a minimal performance degradation (from 3.271 to 3.309). Furthermore, Instantaneous DeVA_{S_∞} yields a marginal improvement over the baseline DeVA_{S_∞} . Notably, in contrast to common findings in image classification tasks, sign-stabilized DeVA_{S_∞} adversely affects the final validation performance.

Algorithm 3 Memory-Efficient DeVA_{S_∞}

Input $\beta_1, \beta_2, \beta_3, \text{freq}, \varepsilon, T, \eta_t$ for $t = 1, \dots, T$

Output $X_T \in \mathbb{R}^{n \times m}$

- 1: Initialize $L_0 \in \mathbb{R}^{n \times n}, R_0 \in \mathbb{R}^{m \times m}, M_0 \in \mathbb{R}^{n \times m}, V_{r,0} \in \mathbb{R}^n, V_{c,0} \in \mathbb{R}^m$
 - 2: Randomly initialize $X_1 \in \mathbb{R}^{n \times m}$
 - 3: **for** $t = 1, \dots, T$ **do**
 - 4: $G_t = \nabla f(X_t, \xi_t)$
 - 5: $L_t = \beta_3 L_{t-1} + (1 - \beta_3) G_t G_t^T$
 - 6: $R_t = \beta_3 R_{t-1} + (1 - \beta_3) G_t^T G_t$
 - 7: **if** $t \bmod \text{freq} == 0$ **then**
 - 8: $Q_L, Q_R = \text{eigdecomp}(L_t), \text{eigdecomp}(R_t)$
 - 9: **end if**
 - 10: $G'_t = Q_L^T G_t Q_R$
 - 11: $M_t = \beta_1 M_{t-1} + (1 - \beta_1) G'_t$
 - 12: $r_t = \left(\sqrt{\sum_{j=1}^m (M_t)_{ij}^2} \right)_{i=1}^n$
 - 13: $c_t = \left(\sqrt{\sum_{i=1}^n (M_t)_{ij}^2} \right)_{j=1}^m$
 - 14: $V_{r,t} = \beta_2 V_{r,t-1} + (1 - \beta_2) r_t$
 - 15: $V_{c,t} = \beta_2 V_{c,t-1} + (1 - \beta_2) c_t$
 - 16: $\Gamma_t = (V_{r,t} V_{c,t}^T \oslash (r_t c_t^T + \varepsilon))^{-1/2}$
 - 17: $D_t = \Gamma_t \odot \text{msign}(M_t) \odot 0.2 \sqrt{\max(n, m)}$
 - 18: $X_{t+1} = X_t - \eta_t D_t$
 - 19: **end for**
-

H.4. Signal-to-noise ratio during pretraining

Algorithm 4 Instantaneous DeVA_{S_∞}**Input** $\beta_1, \beta_2, \beta_3, freq, \varepsilon, T, \eta_t$ for $t = 1, \dots, T$ **Output** $X_T \in \mathbb{R}^{n \times m}$

```

1: Initialize  $L_0 \in \mathbb{R}^{n \times n}, R_0 \in \mathbb{R}^{m \times m}, M_0 \in \mathbb{R}^{n \times m}, V_{r,0} \in \mathbb{R}^n, V_{c,0} \in \mathbb{R}^m$ 
2: Randomly initialize  $X_1 \in \mathbb{R}^{n \times m}$ 
3: for  $t = 1, \dots, T$  do
4:    $G_t = \nabla f(X_t, \xi_t)$ 
5:    $L_t = \beta_3 L_{t-1} + (1 - \beta_3) G_t G_t^T$ 
6:    $R_t = \beta_3 R_{t-1} + (1 - \beta_3) G_t^T G_t$ 
7:   if  $t \bmod freq == 0$  then
8:      $Q_L, Q_R = \text{eigdecomp}(L_t), \text{eigdecomp}(R_t)$ 
9:   end if
10:   $G'_t = Q_L^T G_t Q_R$ 
11:   $M_t = \beta_1 M_{t-1} + (1 - \beta_1) G'_t$ 
12:   $r_t = \left( \sqrt{\sum_{j=1}^m (G'_t)_{ij}^2} \right)_{i=1}^n$ 
13:   $c_t = \left( \sqrt{\sum_{i=1}^n (G'_t)_{ij}^2} \right)_{j=1}^m$ 
14:   $V_t = \beta_2 V_{t-1} + (1 - \beta_2) r_t c_t^T$ 
15:   $\Gamma_t = (V_t \oslash (r_t c_t^T + \varepsilon))^{-1/2}$ 
16:   $D_t = \Gamma_t \odot \text{msign}(M_t) \odot 0.2 \sqrt{\max(n, m)}$ 
17:   $X_{t+1} = X_t - \eta_t D_t$ 
18: end for

```

Table 10. SNR matrix evolution (Frobenius norm). Frobenius norm of the estimated signal-to-noise ratio (SNR) matrix at representative early- and late-depth layers and at the listed pretraining checkpoints. **Boldface** in each row is the largest norm observed for that layer across the five token counts. The norms generally shift quickly within the first few million tokens, consistent with rapid reorganization of gradient statistics while representations are still forming. Thereafter, the norms vary within a comparatively narrow band, consistent with an approximately stationary signal-to-noise balance in late training.

Layer	5M tokens	10M tokens	50M tokens	100M tokens	500M tokens
block.0.mlp.c.proj	1973.9	1634.4	1534.6	1635.8	1652.0
block.0.attn.c.proj	969.5	910.7	753.4	847.5	854.8
block.11.mlp.c.proj	499.1	2367.3	1651.1	1673.4	1747.4
block.11.attn.c.proj	373.4	1210.6	781.2	817.7	894.7